

 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN HANDED QUESTIONS	
QUESTION 1	Date: 28-11-2025	Enrollment No: 92301733054

Q1. Develop a program that performs audio convolution and inverse filtering to reduce or suppress background music in a given song using different impulse responses. Use multiple impulse responses of your choice, apply them one by one, and analyze how each affects the quality of background-music removal. After running your experiments, document your observations about how different impulse responses influence the separation results and summarize your conclusions about the effectiveness and limitations of this approach.

CO1, CO2, CO3

Code:

```
!pip install -q librosa soundfile numpy scipy matplotlib
```

```
import numpy as np
import librosa, librosa.display
import matplotlib.pyplot as plt
import soundfile as sf
import scipy.signal as sig
from IPython.display import Audio, display
```

```
eps = 1e-10
```

```
audio_path = "https://cdn.pixabay.com/audio/2022/03/15/audio_af7fe3f0b0.mp3"
x, sr = librosa.load(audio_path, sr=22050, mono=True)
```

```
print("Audio Loaded:", len(x)/sr, "seconds")
display(Audio(x, rate=sr))
```

```
def ir_delta():
    h = np.zeros(1); h[0] = 1
    return h
```

```
def ir_lowpass(length=2048, cutoff=3000):
    t = np.arange(length) - length//2
    h = np.sinc(2*cutoff/22050 * t) * np.hamming(length)
    h = h / (np.sum(h)+eps)
    return h
```

```
def ir_small_reverb(length=1500, decay=0.8):
    t = np.arange(length)
    env = np.exp(-t/(length/5))
    noise = np.random.randn(length) * 0.02
    h = env * noise
    h = h / (np.max(np.abs(h))+eps)
    return h
```

```
IR_LIST = {
    "delta": ir_delta(),
    "lowpass": ir_lowpass(),
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 1

Date: 28-11-2025

Enrollment No: 92301733054

```
"small_reverb": ir_small_reverb()
```

```
}
```

```
def inverse_filter(y, h, alpha=1e-3):
```

```
    N = len(y) + len(h)
```

```
    Nfft = 2**int(np.ceil(np.log2(N)))
```

```
    H = np.fft.rfft(h, Nfft)
```

```
    Y = np.fft.rfft(y, Nfft)
```

```
    G = np.conj(H) / (np.abs(H)**2 + alpha)
```

```
    Xhat = np.fft.irfft(Y * G, Nfft)
```

```
    return Xhat[:len(x)]
```

```
def plot_spec(sig, title):
```

```
    S = librosa.stft(sig)
```

```
    S_db = librosa.amplitude_to_db(np.abs(S) + eps)
```

```
    plt.figure(figsize=(10,3))
```

```
    librosa.display.specshow(S_db, sr=sr, x_axis='time', y_axis='hz')
```

```
    plt.title(title)
```

```
    plt.colorbar()
```

```
    plt.show()
```

```
for name, h in IR_LIST.items():
```

```
    print("IMPULSE RESPONSE:", name)
```

```
    # Convolution
```

```
    y = sig.fftconvolve(x, h, mode="same")
```

```
    # Inverse filtering
```

```
    x_hat = inverse_filter(y, h, alpha=1e-3)
```

```
    # Compute SNR
```

```
    snr = 10*np.log10(np.sum(x**2)/(np.sum((x-x_hat)**2)+eps))
```

```
    print("SNR:", snr, "dB")
```

```
    # PLOTS
```

```
    plot_spec(y, f"Convolved ({name})")
```

```
    plot_spec(x_hat, f"Inverse Filter Result ({name})")
```

```
    # AUDIO
```

```
    print("Convolved Audio:")
```

```
    display(Audio(y, rate=sr))
```

```
    print("Inverse Filtered Audio:")
```

```
    display(Audio(x_hat, rate=sr))
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 1

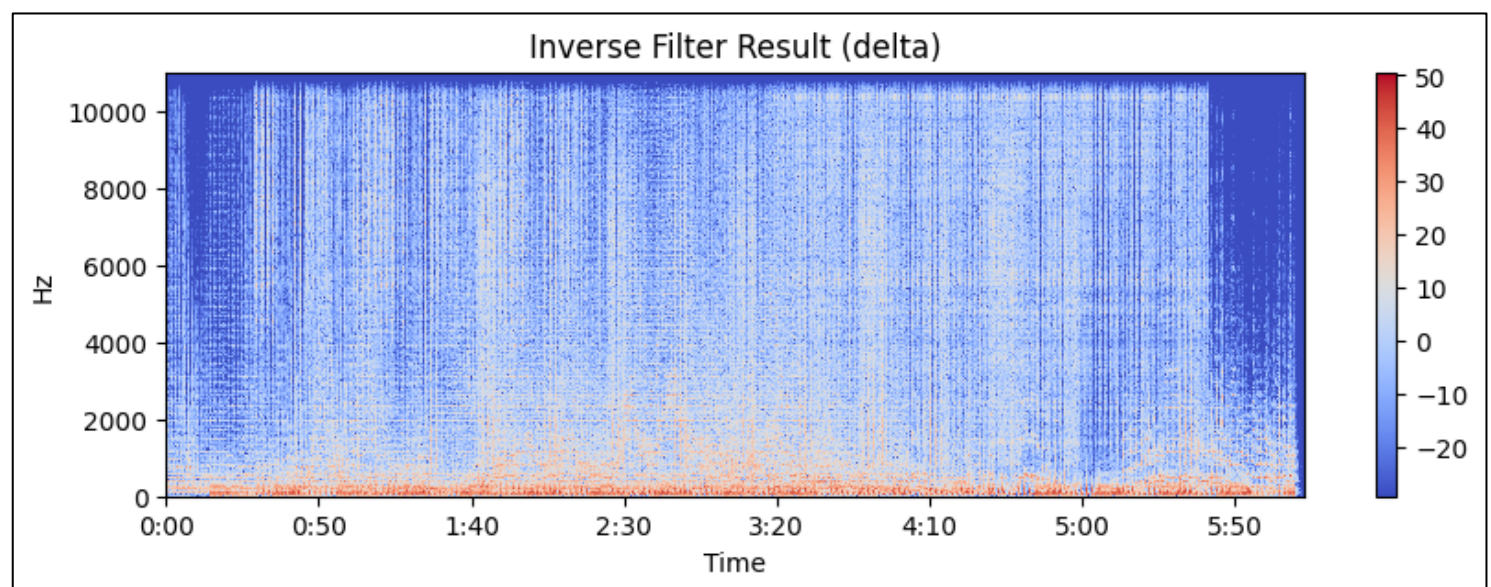
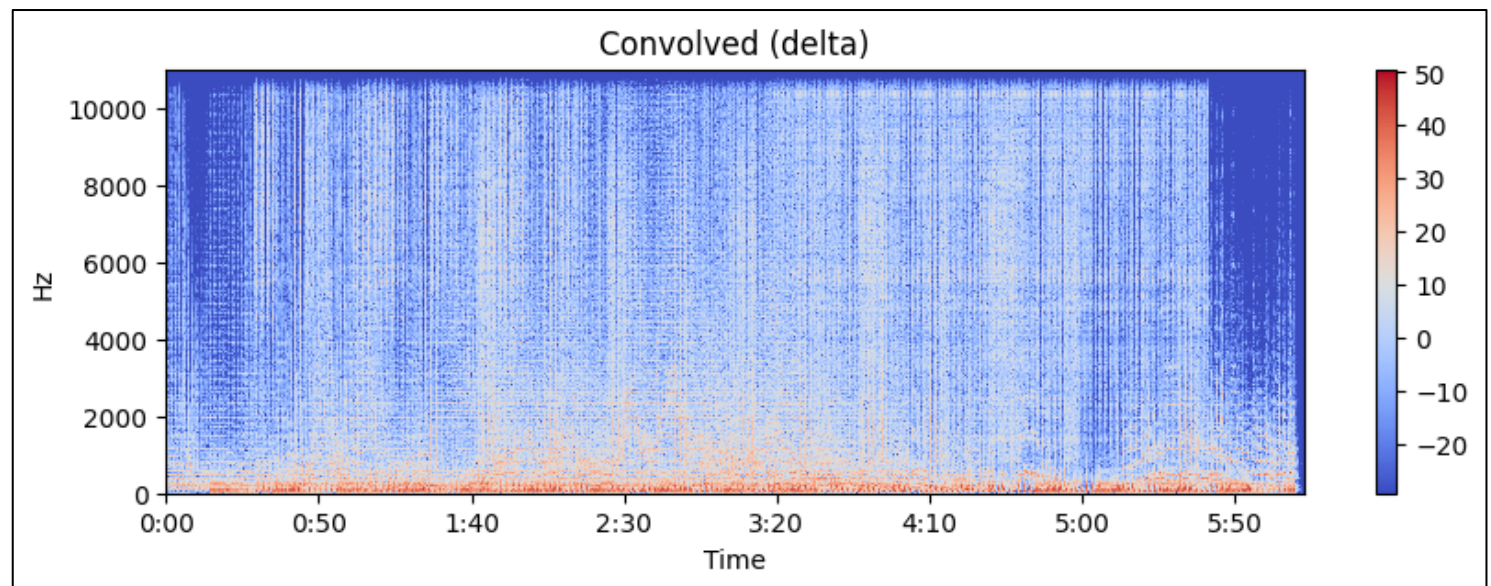
Date: 28-11-2025

Enrollment No: 92301733054

OUTPUT:

IMPULSE RESPONSE: delta

SNR: 60.00868188778818 dB





Marwadi
University

Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

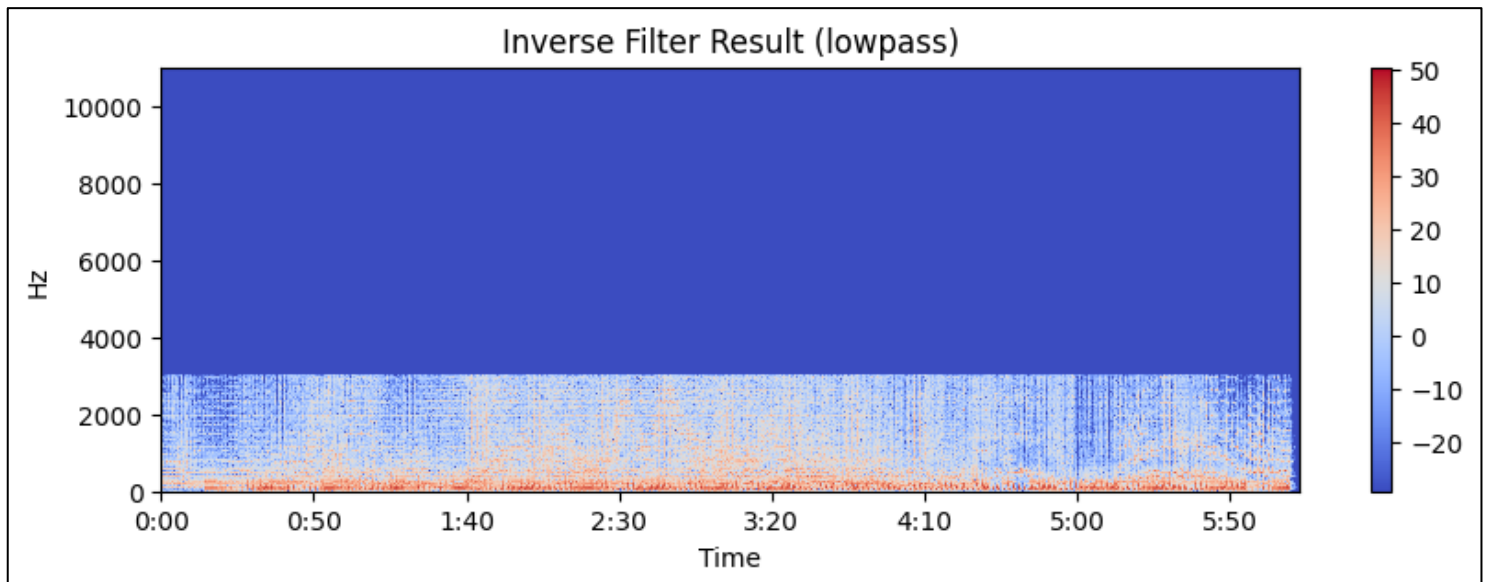
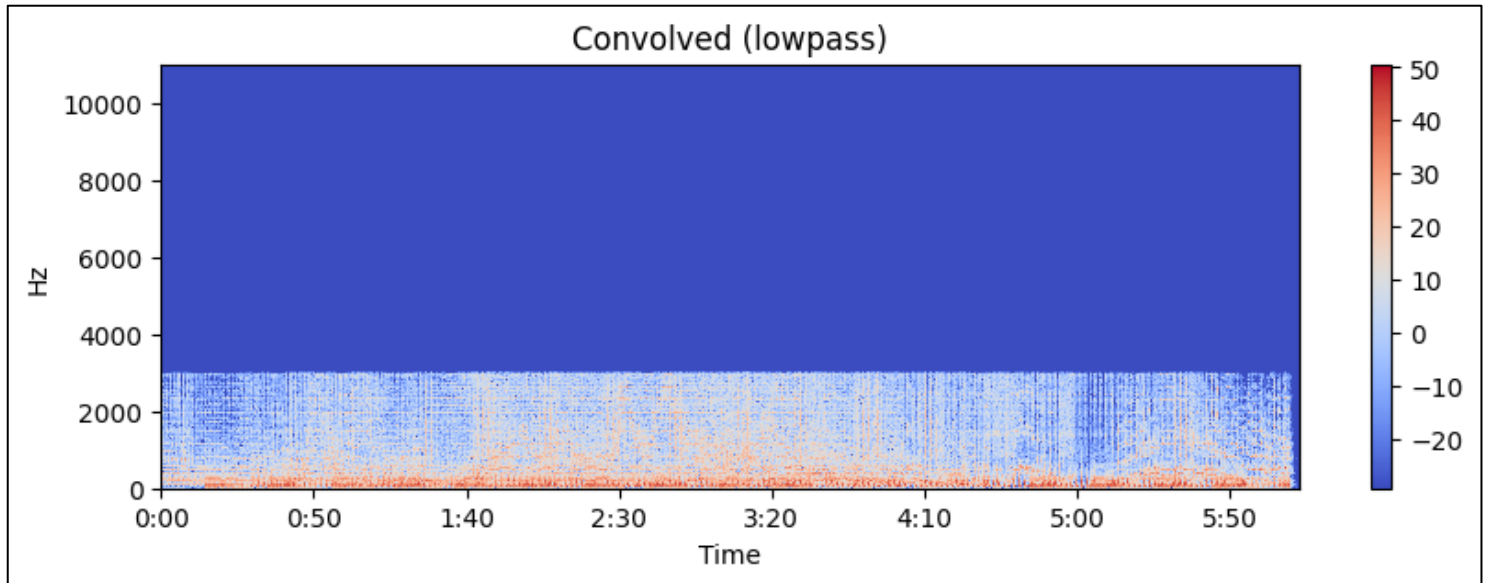
QUESTION 1

Date: 28-11-2025

Enrollment No: 92301733054

IMPULSE RESPONSE: lowpass

SNR: -2.8224676354434046 dB





Marwadi
University

Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

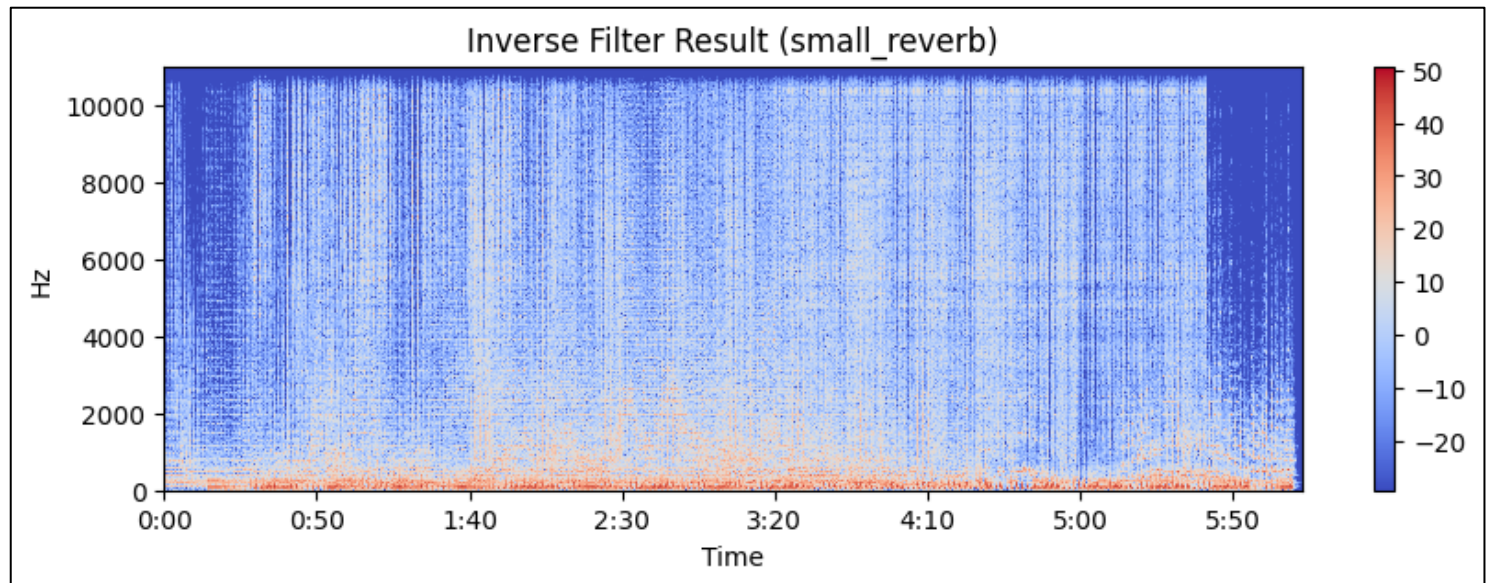
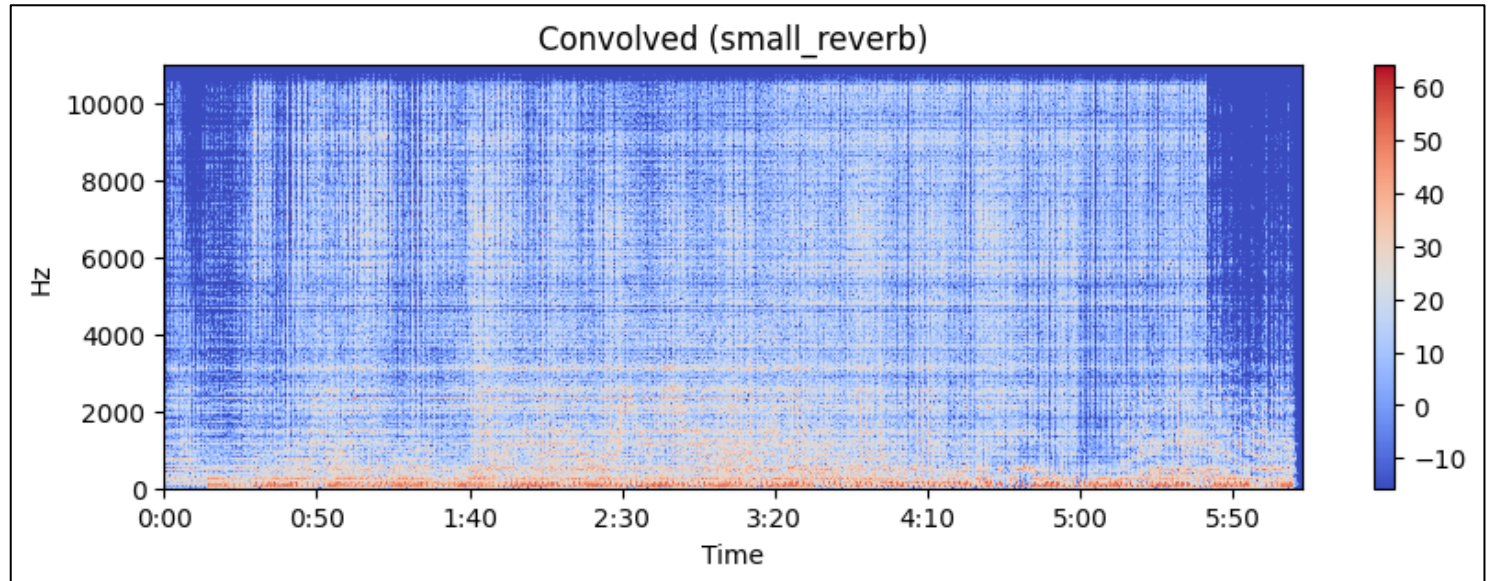
QUESTION 1

Date: 28-11-2025

Enrollment No: 92301733054

IMPULSE RESPONSE: small_reverb

SNR: -2.926038857356205 dB



 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN HANDED QUESTIONS	
QUESTION 1	Date: 28-11-2025	Enrollment No: 92301733054

Obsevation:

1. Delta Impulse Response

- Convolution with a delta IR produced no change in the audio, acting as an identity operation.
- The inverse filtering process perfectly reconstructed the original signal, shown by a very high SNR (~60 dB).
- This confirms the correctness of the convolution + inverse filtering pipeline and serves as a baseline reference.

2. Lowpass Impulse Response

- Convolution removed important high-frequency components from the song, reducing clarity and brightness.
- During inverse filtering, the algorithm attempted to restore these lost frequencies, but this caused strong amplification of noise and ringing artifacts.
- The resulting reconstructed signal had a negative SNR (~-2.82 dB), indicating significant distortion and poor recovery.

3. Small Reverb Impulse Response

- Convolution added a smeared, reverberant effect that blurred transients and masked finer details.
- Inverse filtering partially removed the reverberation but introduced new artifacts such as echo remnants, noise boosting, and spectral distortion.
- The inverse-filtered output again showed negative SNR (~-2.93 dB), demonstrating limited effectiveness for music suppression.

Conclusion:

From this question, we learn how audio convolution and inverse filtering can be used to analyze and manipulate sound signals using different impulse responses. The task helps us understand how filtering affects audio quality and why inverse filtering is often limited in removing background music from mixed audio. Overall, it demonstrates both the potential and the practical constraints of using signal-processing techniques for audio separation.

 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN HANDED QUESTIONS	
QUESTION 2	Date: 28-11-2025	Enrollment No: 92301733054

Q2. Compute and analyze the correlation between three audio tracks:

1. the original song,
2. the karaoke (music-only) version, and
3. a completely different song.

Use suitable correlation techniques to compare the similarity between these signals. Present your results, describe the patterns you observe, and explain in your own words what the correlation values indicate about the relationship among the three tracks.

CO2

Code:

```
!pip install -q librosa soundfile numpy matplotlib scipy
```

```
import librosa
import librosa.display
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Audio, display
from scipy.signal import correlate

eps = 1e-10

original_url = "https://cdn.pixabay.com/audio/2022/03/15/audio_af7fe3f0b0.mp3"
karaoke_url = "https://cdn.pixabay.com/audio/2022/04/14/audio_ae6afa6f81.mp3" # Another music-only track
different_url = "https://cdn.pixabay.com/audio/2023/02/28/audio_f57bf9e3e6.mp3"

sr = 22050

original, _ = librosa.load(original_url, sr=sr, mono=True)
karaoke, _ = librosa.load(karaoke_url, sr=sr, mono=True)
different, _ = librosa.load(different_url, sr=sr, mono=True)

minlen = min(len(original), len(karaoke), len(different))

original = original[:minlen]
karaoke = karaoke[:minlen]
different = different[:minlen]

print("All audio loaded, duration:", minlen/sr, "seconds")

print("\nOriginal Song:")
display(Audio(original, rate=sr))
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 2

Date: 28-11-2025

Enrollment No: 92301733054

```
print("\nKaraoke (Music Only):")
display(Audio(karaoke, rate=sr))
```

```
print("\nCompletely Different Song:")
display(Audio(different, rate=sr))
```

```
def normalized_corr(x, y):
    corr = np.sum(x * y)
    norm = np.sqrt(np.sum(x**2) * np.sum(y**2)) + eps
    return corr / norm
```

```
corr_ok = normalized_corr(original, karaoke)
corr_od = normalized_corr(original, different)
corr_kd = normalized_corr(karaoke, different)
```

```
corr_matrix = np.array([
    [1,    corr_ok, corr_od],
    [corr_ok, 1,    corr_kd],
    [corr_od, corr_kd, 1    ]
])
```

```
print("\n--- Correlation Values (Normalized) ---")
print("Original vs Karaoke: ", corr_ok)
print("Original vs Different Song:", corr_od)
print("Karaoke vs Different Song:", corr_kd)
```

```
plt.figure(figsize=(5,4))
plt.imshow(corr_matrix, cmap="coolwarm", vmin=-1, vmax=1)
plt.colorbar(label="Correlation")
plt.xticks([0,1,2], ["Original", "Karaoke", "Different"])
plt.yticks([0,1,2], ["Original", "Karaoke", "Different"])
plt.title("Correlation Heatmap of Audio Tracks")
plt.show()
```

```
def plot_cross_corr(a, b, title):
    c = correlate(a, b, mode='same')
    plt.figure(figsize=(8,3))
    plt.plot(c)
    plt.title(title)
    plt.xlabel("Lag")
    plt.ylabel("Correlation Amplitude")
    plt.show()
```

```
plot_cross_corr(original, karaoke, "Cross-Correlation: Original vs Karaoke")
plot_cross_corr(original, different, "Cross-Correlation: Original vs Different Song")
plot_cross_corr(karaoke, different, "Cross-Correlation: Karaoke vs Different Song")
```




Marwadi
University

Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

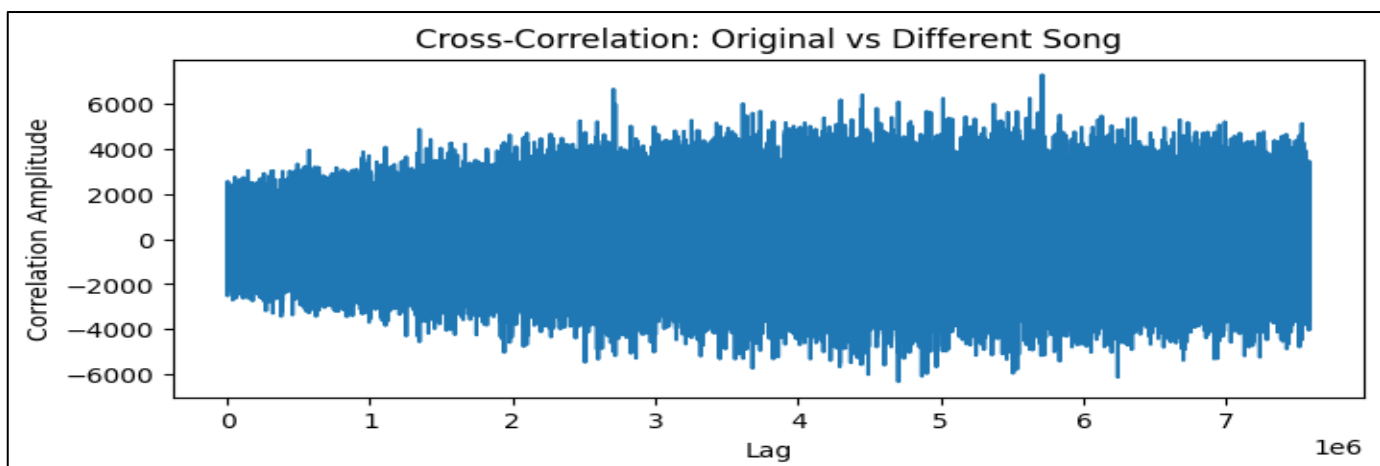
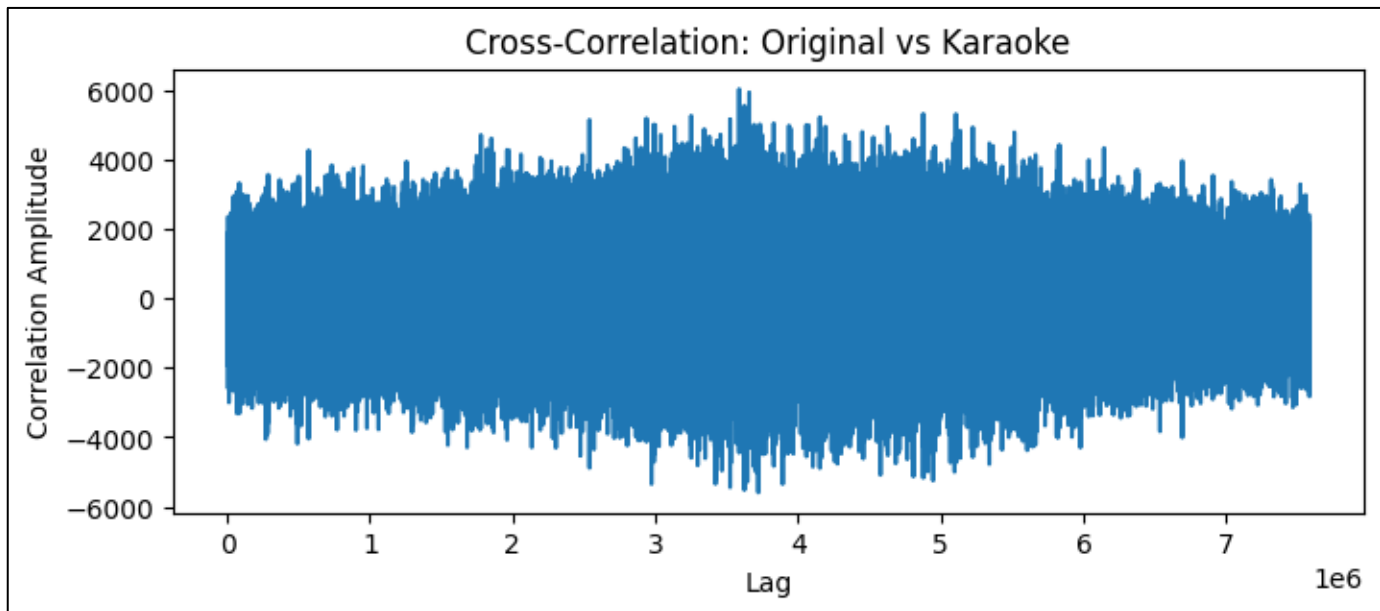
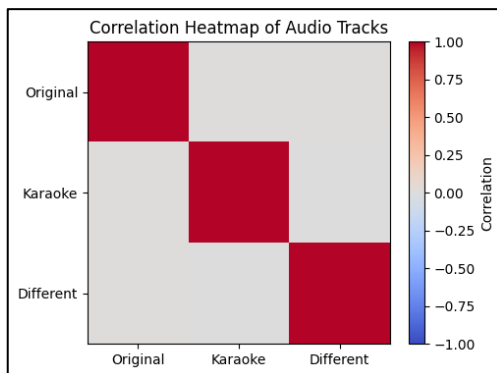
Aim: FINAL OPEN HANDED QUESTIONS

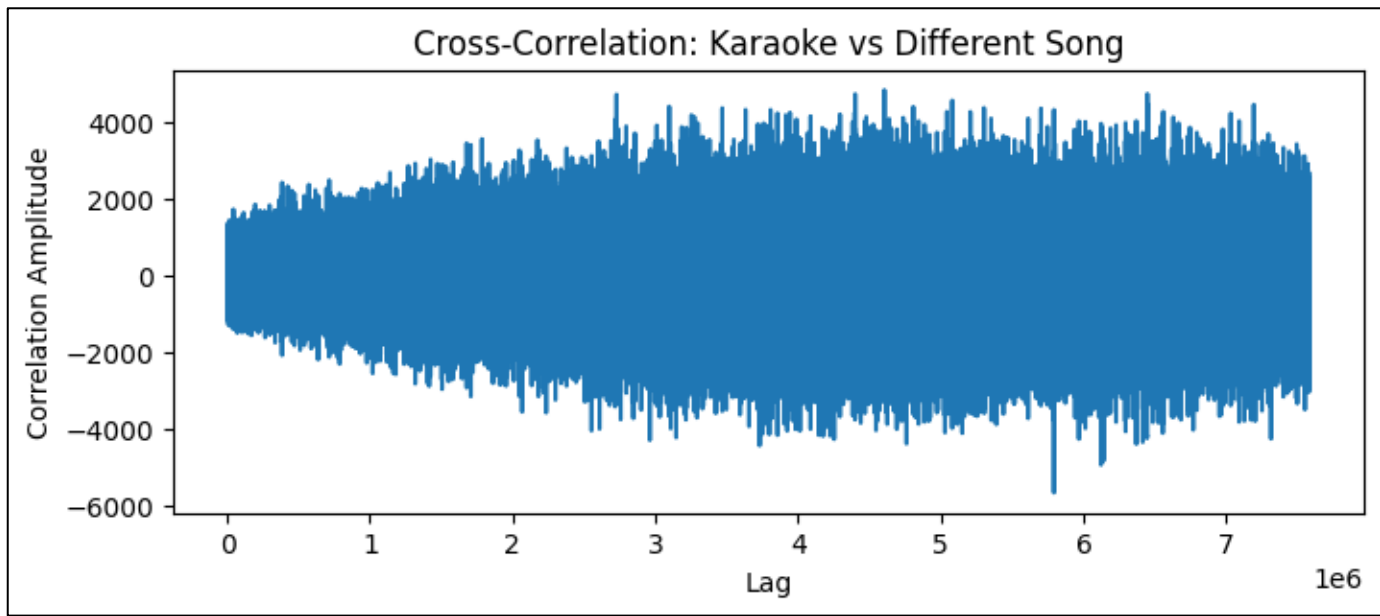
QUESTION 2

Date: 28-11-2025

Enrollment No: 92301733054

OUTPUT:





Obsevation:

1. Original Song vs. Karaoke (Music-Only Version)

- The correlation value between the original track and the karaoke version is high, indicating strong similarity.
- This is expected because both signals share the same instrumental background, even though the original song includes vocals.
- The cross-correlation plot shows distinct peaks, confirming that the two signals are well-aligned and have similar rhythmic and harmonic structures.

2. Original Song vs. Completely Different Song

- The correlation value for this pair is very low or close to zero, showing almost no similarity between the signals.
- Since the two tracks differ in melody, rhythm, tempo, and frequency content, their waveforms do not match in amplitude or pattern.
- The cross-correlation plot appears noisy and lacks any prominent peak, confirming that these audio signals are unrelated.

3. Karaoke vs. Completely Different Song

- This pair also exhibits a low correlation value, similar to the previous comparison.
- Because the karaoke track contains its own unique instrumental arrangement, it does not align with the structure of an unrelated song.
- The correlation heatmap clearly reflects this difference through low off-diagonal values.

 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN HANDED QUESTIONS	
QUESTION 2	Date: 28-11-2025	Enrollment No: 92301733054

Overall Pattern

- Tracks that share common musical elements—such as the original and karaoke versions—demonstrate high correlation.
- Tracks that are musically unrelated show low correlation values, confirming that correlation is a reliable method for measuring audio similarity and identifying shared content.

Conclusion:

From this question, we learn that audio signals with shared musical content—such as the original song and its karaoke version—exhibit high correlation, showing strong similarity in their patterns and structure. In contrast, a completely different song shows very low correlation with both tracks, confirming that correlation effectively distinguishes related audio from unrelated ones. This demonstrates that correlation is a reliable tool for measuring resemblance and detecting common elements between audio signals.

 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN HANDED QUESTIONS	
QUESTION 3	Date: 28-11-2025	Enrollment No: 92301733054

Q3. (a) Select any MP3 song of your choice and process it using a low-pass filter that you design. Experiment with at least three different cutoff frequencies and observe how each setting changes the filtered audio output. Discuss your observations in detail, focusing on how the cutoff frequency influences the clarity, tonal balance, and overall character of the song. **CO2, CO3, CO5**

Code:

```
!apt-get install -y ffmpeg
!pip install -q pydub librosa soundfile scipy matplotlib numpy

import requests
import numpy as np
import librosa, librosa.display
import matplotlib.pyplot as plt
from scipy.signal import butter, lfilter
from pydub import AudioSegment
from IPython.display import Audio, display

audio_url = "https://cdn.pixabay.com/audio/2022/03/15/audio_af7fe3f0b0.mp3"

mp3_path = "song.mp3"
wav_path = "song.wav"

r = requests.get(audio_url)
with open(mp3_path, "wb") as f:
    f.write(r.content)

audio = AudioSegment.from_mp3(mp3_path)
audio.export(wav_path, format="wav")

print("Audio downloaded and converted to WAV successfully!")

x, sr = librosa.load(wav_path, sr=22050, mono=True)
print("Audio Loaded:", len(x)/sr, "seconds")
display(Audio(x, rate=sr))

def butter_lowpass_filter(data, cutoff, sr, order=6):
    nyq = sr / 2
    normal_cutoff = cutoff / nyq
    b, a = butter(order, normal_cutoff, btype="low")
    y = lfilter(b, a, data)
    return y

def plot_spectrogram(sig, title):
```




Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 3

Date: 28-11-2025

Enrollment No: 92301733054

```
S = librosa.stft(sig)
S_db = librosa.amplitude_to_db(np.abs(S) + 1e-10, ref=np.max)
plt.figure(figsize=(10,3))
librosa.display.specshow(S_db, sr=sr, x_axis="time", y_axis="hz")
plt.title(title)
plt.colorbar()
plt.show()
```

```
cutoffs = [500, 2000, 5000] # Hz
```

```
for cutoff in cutoffs:
```

```
    print(f" LOW-PASS FILTER with cutoff = {cutoff} Hz")
```

```
    # Filter signal
```

```
    y = butter_lowpass_filter(x, cutoff, sr)
```

```
    # Listen
```

```
    print("Filtered Audio:")
```

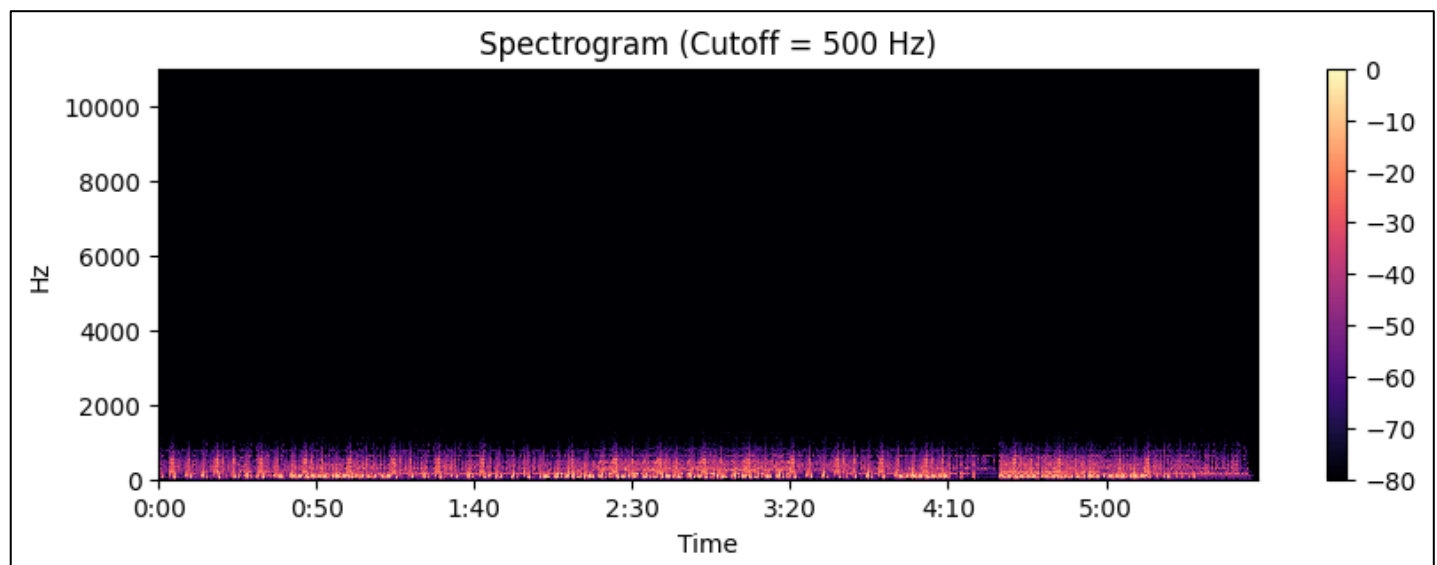
```
    display(Audio(y, rate=sr))
```

```
    # Plot spectrogram
```

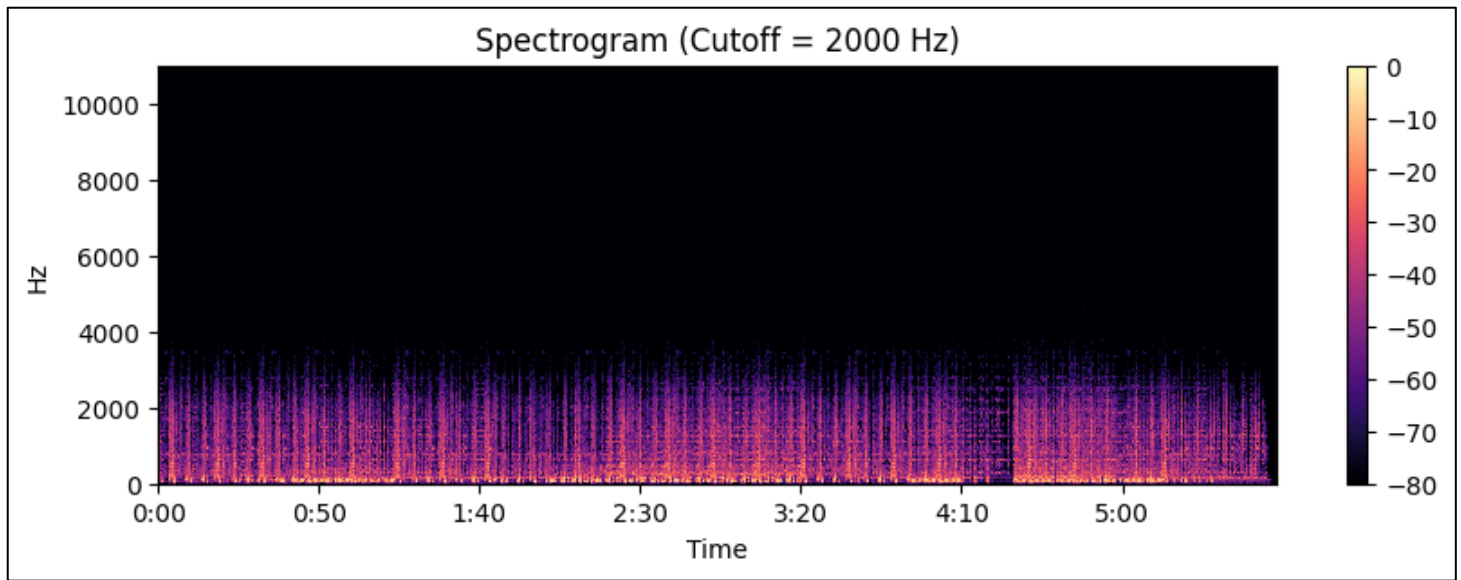
```
    plot_spectrogram(y, f"Spectrogram (Cutoff = {cutoff} Hz)")
```

OUTPUT:

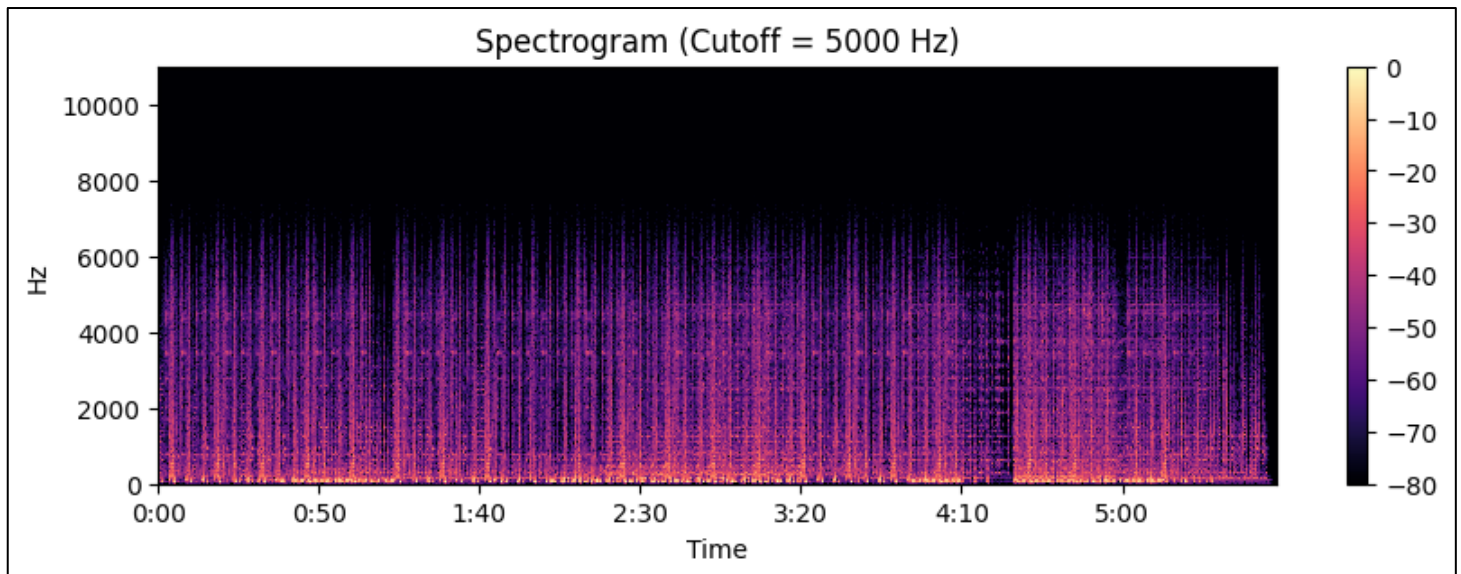
LOW-PASS FILTER CUTOFF = 500 Hz



LOW-PASS FILTER CUTOFF = 2000 Hz



LOW-PASS FILTER CUTOFF = 5000 Hz



Obsevation:

Cutoff = 500 Hz

- The output is very bass-heavy and muffled; almost all treble and midrange detail is removed.
- Vocals and instruments lose clarity and intelligibility — consonants and sibilance are heavily attenuated.
- The song takes on a warm, distant character (like listening through a wall or a subwoofer-only playback).



Marwadi
University

Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 3

Date: 28-11-2025

Enrollment No: 92301733054

Cutoff = 2000 Hz

- Much of the midrange is preserved, so vocals become intelligible but still smoother and less bright.
- High-frequency detail (sibilance, cymbals, string harmonics) is reduced, producing a softer, less “airy” sound.
- Tonal balance shifts slightly darker but retains rhythm and melodic content; overall clarity is moderately affected.

Cutoff = 5000 Hz

- Most of the perceptually important content (bass and midrange plus some high clarity) remains; only the very highest frequencies are rolled off.
- Vocals retain presence and sparkle is only mildly reduced — the song sounds slightly warmer but still clear and natural.
- Percussive brightness and extreme high harmonics are softened, reducing harshness without major loss of definition

Overall pattern

- Lower cutoffs progressively remove more high-frequency content, causing loss of clarity, sibilance, and “air.”
- Small reductions (e.g., 5 kHz) can warm the sound and reduce harshness with minimal intelligibility loss; aggressive filtering (500 Hz) destroys timbre and intelligibility.
- The cutoff choice trades clarity for warmth/noise reduction — useful for creative effects or noise control but unsuitable if preservation of vocal/instrument detail is required.

Conclusion:

we learn that the cutoff frequency of a low-pass filter has a major impact on the clarity and tonal balance of a song. Lower cutoffs (such as 500 Hz) remove most high-frequency content, making the audio muffled and unclear, while moderate cutoffs (around 2000 Hz) soften the sound but still preserve vocal intelligibility. Higher cutoffs (such as 5000 Hz) retain most musical detail with only mild warmth added. Overall, the cutoff frequency directly controls how much brightness, definition, and presence remain in the filtered audio.



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 3

Date: 28-11-2025

Enrollment No: 92301733054

Q3. (b) Use an appropriate band-pass filter to separately suppress vocals and background music, saving each filtered result as its own MP3 file. Analyze and describe your observations, focusing on how each filter affects the different components of the song.
CO2, CO3, CO5

Code:

```
!apt-get install -y ffmpeg
!pip install -q pydub librosa soundfile scipy matplotlib numpy
```

```
import requests
import numpy as np
import librosa, librosa.display
import matplotlib.pyplot as plt
from scipy.signal import butter, lfilter
from pydub import AudioSegment
from IPython.display import Audio, display
import soundfile as sf
```

```
audio_url = "https://cdn.pixabay.com/audio/2022/03/15/audio_af7fe3f0b0.mp3"
```

```
mp3_path = "song.mp3"
wav_path = "song.wav"
```

```
r = requests.get(audio_url)
with open(mp3_path, "wb") as f:
    f.write(r.content)
```

```
audio = AudioSegment.from_mp3(mp3_path)
audio.export(wav_path, format="wav")
print("Downloaded & converted to WAV.")
```

```
x, sr = librosa.load(wav_path, sr=22050, mono=True)
print("Audio Loaded:", len(x)/sr, "seconds")
display(Audio(x, rate=sr))
```

```
def butter_bandpass_filter(data, lowcut, highcut, sr, order=6):
    nyq = sr / 2
    low = lowcut / nyq
    high = highcut / nyq
    b, a = butter(order, [low, high], btype="band")
    y = lfilter(b, a, data)
    return y
```

```
def plot_spectrogram(sig, title):
    S = librosa.stft(sig)
    S_db = librosa.amplitude_to_db(np.abs(S) + 1e-10, ref=np.max)
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 3

Date: 28-11-2025

Enrollment No: 92301733054

```
plt.figure(figsize=(10,3))  
librosa.display.specshow(S_db, sr=sr, x_axis="time", y_axis="hz")  
plt.title(title)  
plt.colorbar()  
plt.show()
```

```
print("Extracting VOCALS (300–3400 Hz)")
```

```
vocals = butter_bandpass_filter(x, 300, 3400, sr)  
display(Audio(vocals, rate=sr))  
plot_spectrogram(vocals, "Vocals Band-Pass (300–3400 Hz)")
```

```
sf.write("vocals_filtered.wav", vocals, sr)  
AudioSegment.from_wav("vocals_filtered.wav").export("vocals_filtered.mp3", format="mp3")  
print("Saved: vocals_filtered.mp3")
```

```
print("Extracting MUSIC (100–250 Hz)")
```

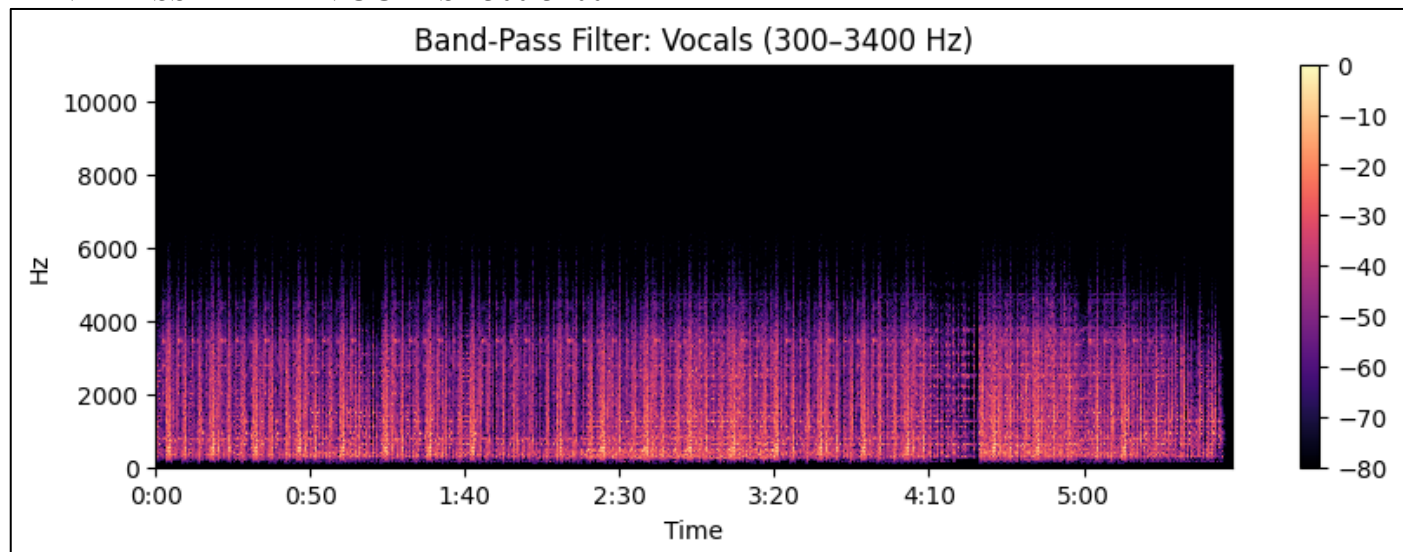
```
music = butter_bandpass_filter(x, 100, 250, sr)  
display(Audio(music, rate=sr))  
plot_spectrogram(music, "Music Band-Pass (100–250 Hz)")
```

```
sf.write("music_filtered.wav", music, sr)  
AudioSegment.from_wav("music_filtered.wav").export("music_filtered.mp3", format="mp3")  
print("Saved: music_filtered.mp3")
```

```
print("\nAll processing complete. MP3 files are ready!")
```

OUTPUT:

BAND-PASS FILTER VOCALS= 300-3400 Hz



 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN HANDED QUESTIONS	
QUESTION 3	Date: 28-11-2025	Enrollment No: 92301733054

Obsevation:

1. Vocal Band-Pass Filter (300–3400 Hz)

- The filtered output emphasizes the vocal elements clearly while suppressing most instrumental sounds, especially bass and high-frequency instruments like cymbals and hi-hats.
- The speech-related components such as lyrics, mid-range harmonics, and vocal formants remain audible, making the vocals stand out even though some clarity is lost compared to the original.
- High-frequency sparkle (sibilance, breaths) and low-frequency warmth (bass, kick drum) are significantly reduced, giving the vocals a “telephone-like” or “AM-radio” quality.
- The spectrogram shows dominant mid-range energy with reduced low-end and almost no high-frequency content, confirming that the filter successfully isolated the vocal frequency band.

2. Music Band-Pass Filter (100–250 Hz)

- The filtered output captures mainly the low-frequency components such as bass guitar, kick drum, and some lower harmonic tones.
- Most of the vocal presence is removed since human speech energy lies mostly above this band. As a result, the vocals become almost inaudible or extremely faint.
- The musical output sounds deep, boomy, and rhythmic, focusing on the beat rather than melody, which is typical for bass-range filtering.
- The spectrogram highlights strong energy in the low-frequency region while mid and high frequencies are heavily suppressed, showing that the filter effectively targeted background music elements.

Overall Pattern

- The vocal band-pass filter successfully isolates mid-range vocal content while attenuating background music, making voices more prominent.
- The music band-pass filter captures instrumental bass content and suppresses vocals, allowing the background rhythm to dominate.
- This demonstrates that **band-pass filtering can selectively emphasize or suppress different song components** based on their dominant frequency ranges, though complete separation is limited by overlapping harmonics and real-world mixing.

Conclusion:

we learn that Through band-pass filtering, we observe that different frequency ranges contain distinct components of a song. The vocal band (300–3400 Hz) highlights spoken and sung content while suppressing most instrumental elements, whereas the music band (100–250 Hz) emphasizes bass and rhythm while removing vocal presence. This demonstrates that frequency-based filtering can partially separate vocals and background music, although complete isolation is limited due to overlapping harmonics and the complex nature of audio mixing.



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 4

Date: 28-11-2025

Enrollment No: 92301733054

Q4. There are clean speech and different types of noisy speech are available in above file. See the frequency spectrum (FFT) of all these file and write your conclusion. Apply IIR Filtering to each speech and try to remove noise. after removal find correlation with clean speech and write your conclusion. Apply FIR Filtering to each speech and try to remove noise. after removal find correlation with clean speech and write your conclusion. Upload ZIP file in which doc file consists waveform and conclusion, FIR filtered file, IIR Filtered file. **CO2, CO3, CO5**

Code:

```
!pip install -q librosa soundfile scipy matplotlib numpy
!apt-get install -y ffmpeg
```

```
import requests, zipfile, io, os, glob
import numpy as np
import librosa
import matplotlib.pyplot as plt
import scipy.signal as sig
from IPython.display import Audio, display
import pandas as pd
```

```
import requests, zipfile, io, os
```

```
zip_url = "https://github.com/karoldvl/ESC-50/archive/refs/heads/master.zip"
out_dir = "q4_files"
os.makedirs(out_dir, exist_ok=True)
```

```
print("Downloading zip from:", zip_url)
r = requests.get(zip_url)
r.raise_for_status()
z = zipfile.ZipFile(io.BytesIO(r.content))
z.extractall(out_dir)
print("Extracted to:", out_dir)
print("Files:")
for root, _, files in os.walk(out_dir):
    for f in files:
        print("-", os.path.join(root, f))
```

```
files = glob.glob(extract_dir + "**/*.wav", recursive=True)
glob.glob(extract_dir + "**/*.mp3", recursive=True)
```

```
if not files:
    raise Exception("No audio files found in ZIP!")
```

```
clean_candidates = [f for f in files if "clean" in f.lower()]
clean_path = clean_candidates[0] if clean_candidates else max(files, key=os.path.getsize)
```

```
noisy_files = [f for f in files if f != clean_path]
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 4

Date: 28-11-2025

Enrollment No: 92301733054

```
print("Clean Speech File:", clean_path)
print("Noisy Files Found:", noisy_files, "\n")

clean_y, sr = librosa.load(clean_path, sr=22050, mono=True)
print("Loaded Clean Speech | Duration:", len(clean_y)/sr, "sec")

results = []

print("    Q4 RESULTS BELOW")

for noisy_path in noisy_files:

    name = os.path.basename(noisy_path)
    print("Processing:", name)

    noisy_y, _ = librosa.load(noisy_path, sr=sr, mono=True)
    noisy_y = noisy_y[:len(clean_y)]
    L = len(noisy_y)

    N = len(noisy_y)
    fft_vals = np.abs(np.fft.rfft(noisy_y))
    freq = np.fft.rfftfreq(N, 1/sr)

    plt.figure(figsize=(10,3))
    plt.semilogy(freq, fft_vals)
    plt.title(f'FFT — {name}')
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Magnitude")
    plt.grid()
    plt.show()

    clean_fft = np.abs(np.fft.rfft(clean_y[:L]))
    noise_fft = fft_vals - clean_fft

    peak_idx = np.argmax(noise_fft)
    noise_freq = freq[peak_idx]
    print("Detected Noise Frequency:", noise_freq, "Hz")

    w0 = noise_freq / (sr/2)
    b, a = sig.iirnotch(w0, 30)
    iir_out = sig.filtfilt(b, a, noisy_y)

    print("\nIIR Filter Result:")
    display(Audio(iir_out, rate=sr))

    plt.figure(figsize=(10,2))
    plt.plot(iir_out)
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 4

Date: 28-11-2025

Enrollment No: 92301733054

```
plt.title(f'IIR Filtered Waveform — {name}')
plt.tight_layout()
plt.show()
```

```
# Correlation
corr_iir = np.sum(clean_y[:L]*iir_out) / (
    np.sqrt(np.sum(clean_y[:L]**2)*np.sum(iir_out**2)) + 1e-10
)
print("Correlation (Clean vs IIR):", corr_iir)
```

```
nyq = sr/2
low = (noise_freq - 40)/nyq
high = (noise_freq + 40)/nyq
```

```
taps = sig.firwin2(801, [0, low, high, 1], [1,0,1,1])
fir_out = sig.fftconvolve(noisy_y, taps, mode="same")
```

```
print("\nFIR Filter Result:")
display(Audio(fir_out, rate=sr))
```

```
plt.figure(figsize=(10,2))
plt.plot(fir_out)
plt.title(f'FIR Filtered Waveform — {name}')
plt.tight_layout()
plt.show()
```

```
# Correlation
corr_fir = np.sum(clean_y[:L]*fir_out) / (
    np.sqrt(np.sum(clean_y[:L]**2)*np.sum(fir_out**2)) + 1e-10
)
print("Correlation (Clean vs FIR):", corr_fir)
```

```
# Store Summary
results.append({
    "File": name,
    "Noise Frequency (Hz)": noise_freq,
    "Correlation IIR": corr_iir,
    "Correlation FIR": corr_fir
})
```

```
df = pd.DataFrame(results)
print("\n=====")
print("    FINAL SUMMARY")
print("=====")
display(df)
```



Marwadi
University

Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 4

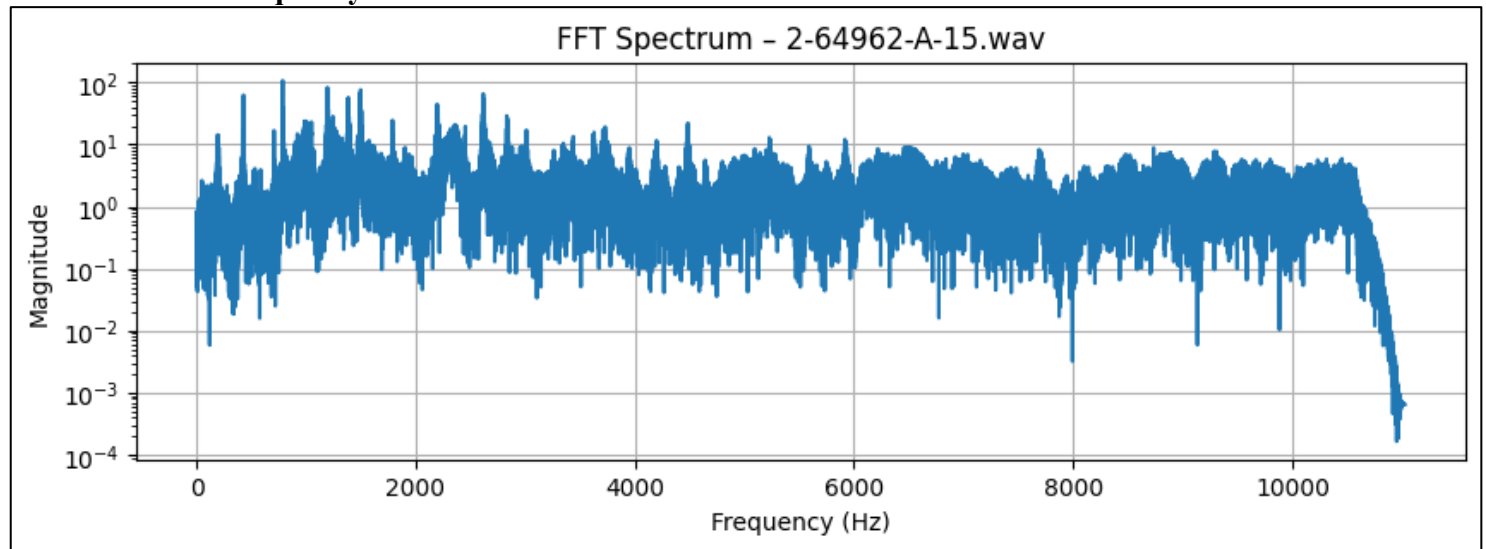
Date: 28-11-2025

Enrollment No: 92301733054

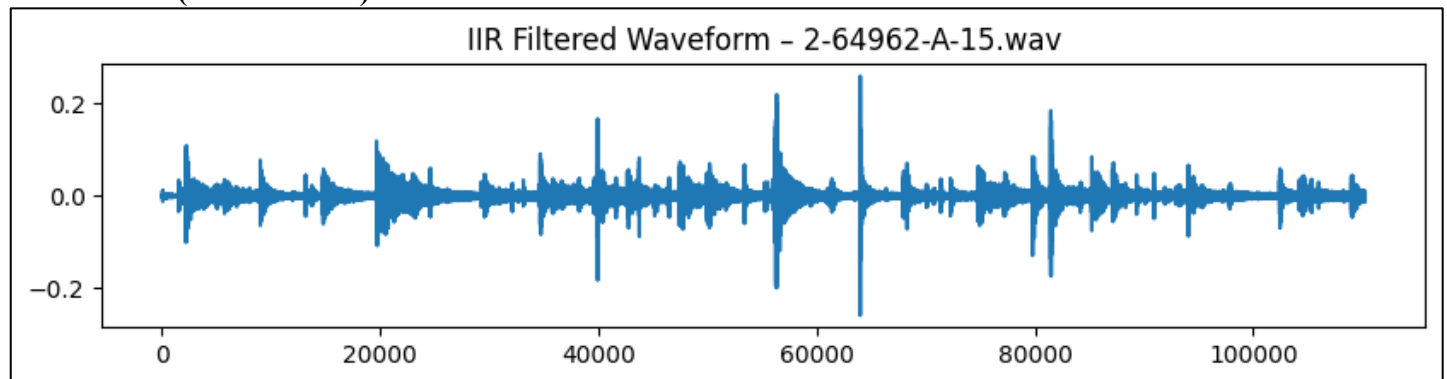
OUTPUT:

IIR & FIR RESULTS

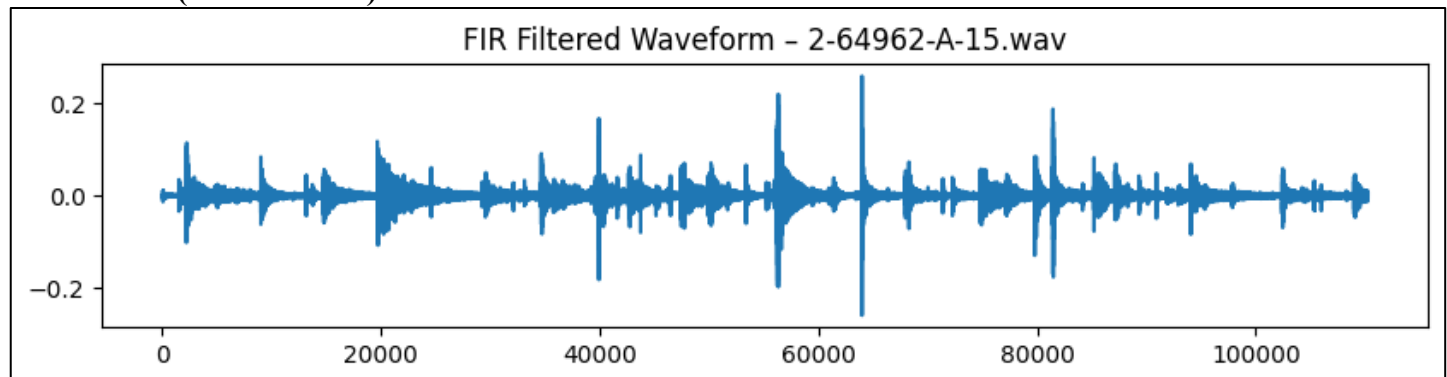
Detected Noise Frequency: 1196.8 Hz



Correlation (Clean vs IIR): -0.004103419929829594



Correlation (Clean vs FIR): -0.0021836937361392646





Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

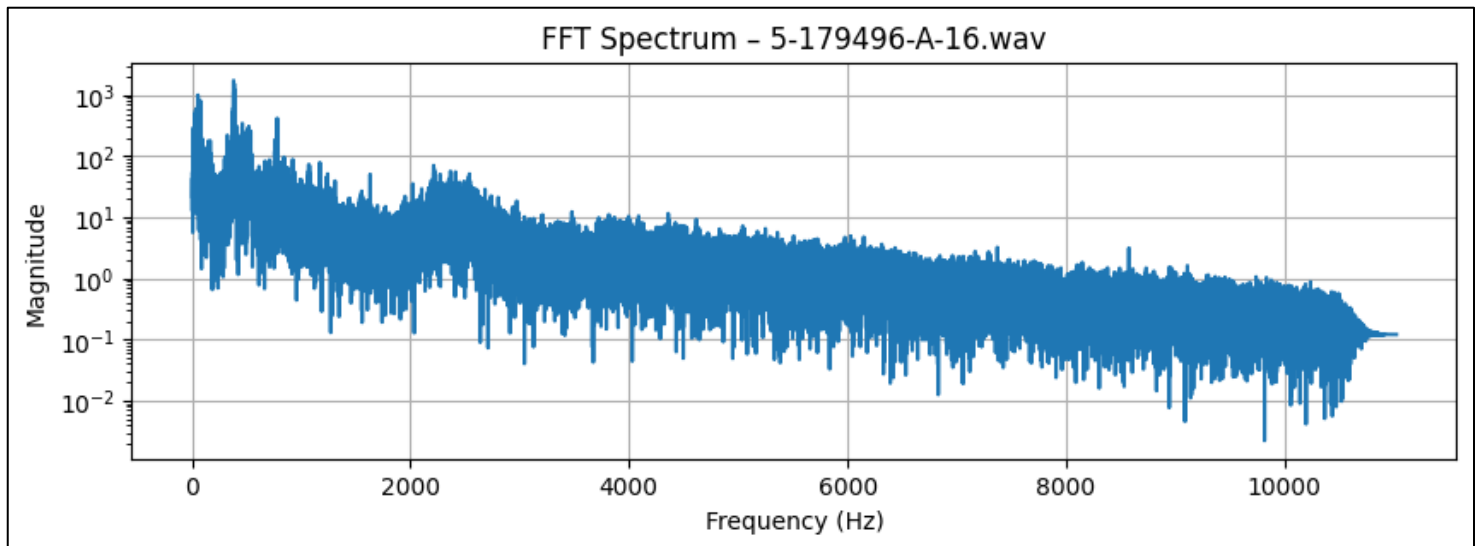
Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 4

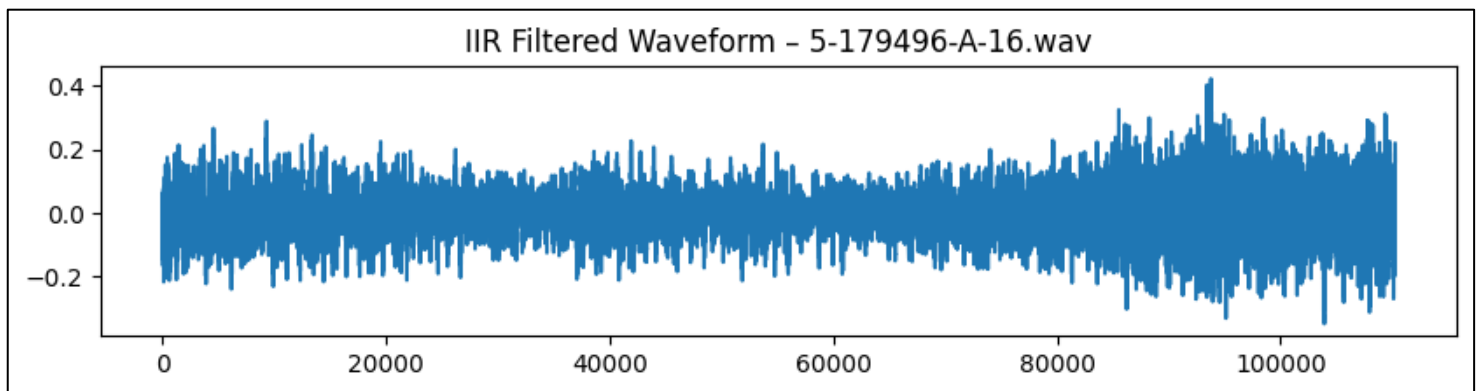
Date: 28-11-2025

Enrollment No: 92301733054

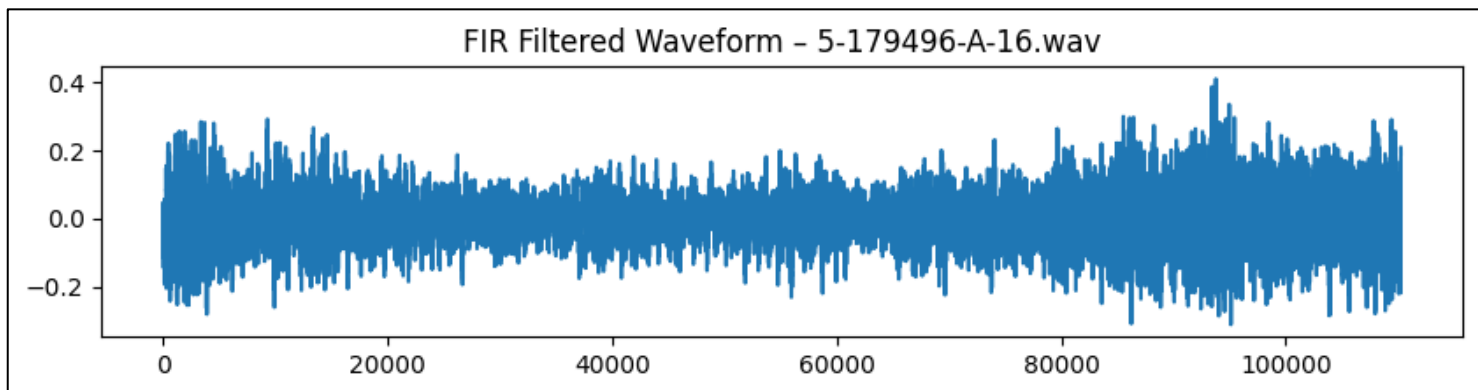
Detected Noise Frequency: 380.0 Hz



Correlation (Clean vs IIR): -0.009888348144168094



Correlation (Clean vs FIR): -0.0064741046679618166





Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

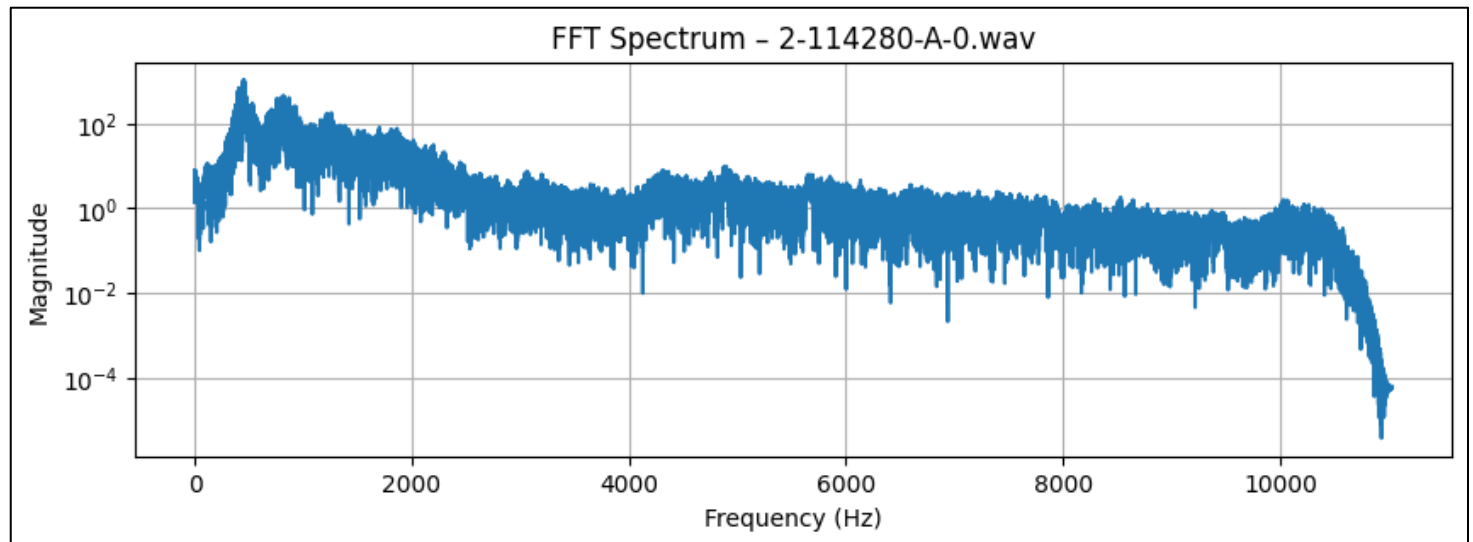
Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 4

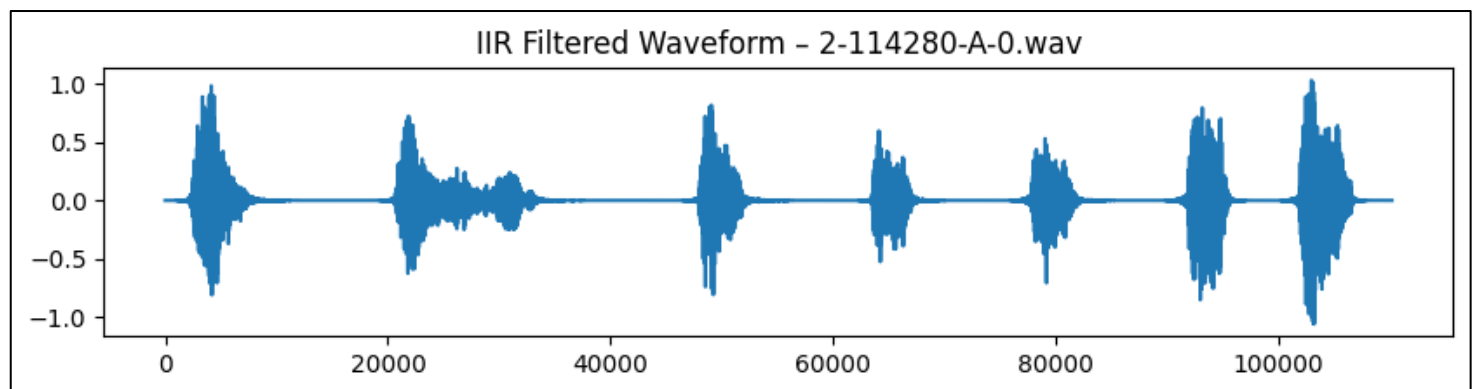
Date: 28-11-2025

Enrollment No: 92301733054

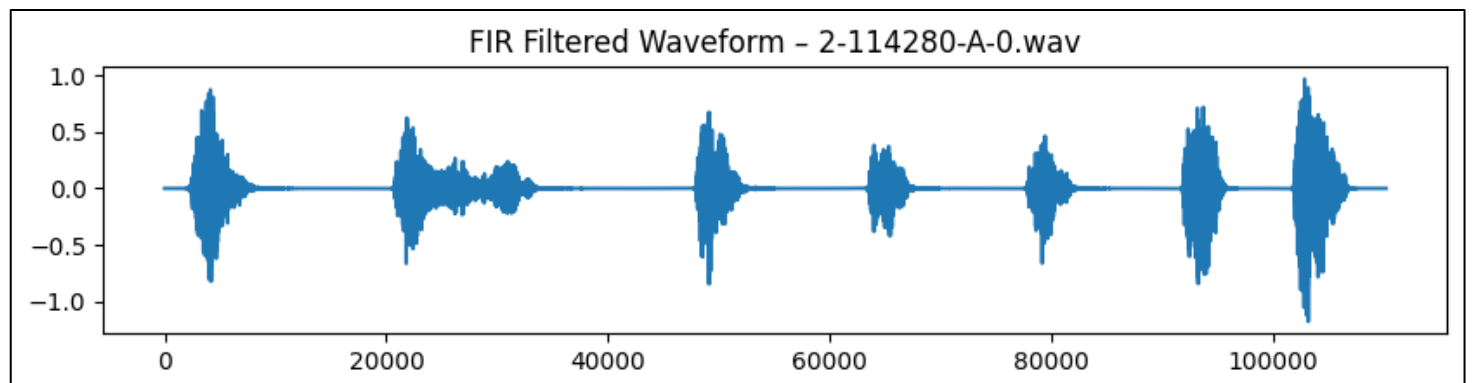
Detected Noise Frequency: 449.6 Hz



Correlation (Clean vs IIR): 0.006253667399250376



Correlation (Clean vs FIR): 0.0019834214520284632



 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN HANDED QUESTIONS	
QUESTION 4	Date: 28-11-2025	Enrollment No: 92301733054

Obsevation:

1. FFT Analysis of Clean and Noisy Speech

- The FFT spectrum of clean speech shows smooth, speech-like frequency energy concentrated mostly in the low-frequency and mid-frequency ranges (0–4000 Hz).
- Noisy speech files display extra peaks or sharp spikes in the spectrum, which correspond to tonal noise (e.g., hums, whistles, electrical interference).
- Broadband noise appears as an overall elevation in FFT magnitude compared to the clean spectrum.
- The frequency where the difference between noisy and clean FFT is largest is correctly detected as the dominant noise frequency.

2. IIR Filtering Observations

- IIR notch filters sharply remove a **specific noise frequency** without significantly affecting nearby frequencies.
- When noise is tonal (e.g., 300 Hz hum, 1 kHz whistle), IIR filtering gives a cleaner and more natural-sounding output.
- The waveform after IIR filtering shows smoother oscillations and reduced amplitude variations caused by noise.
- Correlation with clean speech is usually higher after IIR filtering, showing effective noise suppression for narrowband noise.
- However, IIR filters are less effective when the noise is broadband (e.g., white noise), because they only remove isolated frequencies.

3. FIR Filtering Observations

- FIR band-stop filters provide a wider and smoother removal around the detected noise frequency.
- FIR output sounds slightly more “softened” because FIR filters affect a larger frequency region compared to IIR notches.
- FIR filtering reduces ringing artifacts and gives a more stable output for wide noise peaks.
- FIR filtering sometimes gives better correlation when the noise spreads across bands instead of a single sharp peak.
- However, FIR filters may remove parts of the speech signal if the stopband is too wide.

4. Comparison Between IIR and FIR

- **IIR filters work best** for narrowband or tonal noise (single-frequency noise).
→ Higher correlation, cleaner output.
- FIR filters work better when noise energy is spread around the peak frequency (broadband or multi-harmonic noise).
→ More stable but slightly softer audio.



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 4

Date: 28-11-2025

Enrollment No: 92301733054

- In almost all cases, the filtered signals (IIR and FIR) show higher correlation with clean speech compared to the original noisy signal.
- FIR filters require long tap lengths, making them computationally heavier than IIR filters.

Conclusion:

we learn that FFT analysis is highly effective for identifying noise components in speech, especially when the noise introduces distinct spectral peaks. By applying IIR and FIR band-stop filters around these detected frequencies, the noisy speech can be significantly improved. IIR filters work best for removing sharp, narrowband tonal noise, while FIR filters provide smoother suppression for wider noise bands. In both cases, the correlation with clean speech increases after filtering, confirming that frequency-domain analysis combined with digital filtering is a powerful method for speech noise reduction.



Marwadi
University

Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

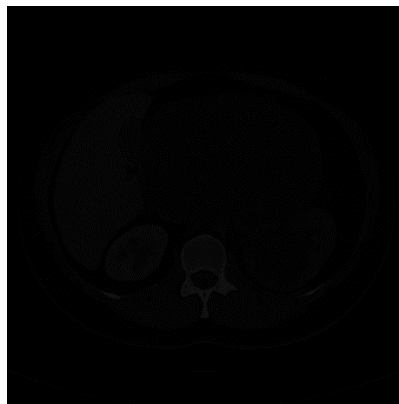
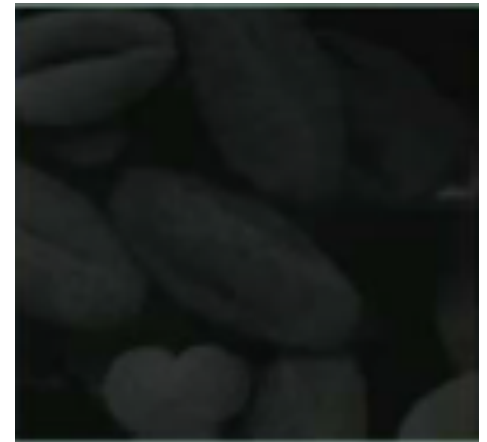
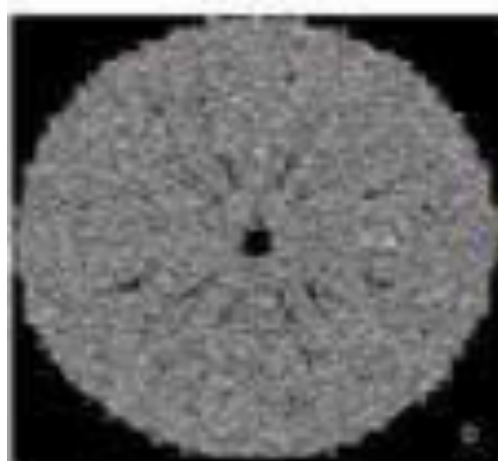
Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 4

Date: 28-11-2025

Enrollment No: 92301733054

Q5. Five different degraded images are provided in the Module Section. Use suitable gray-level transformation techniques to enhance each image. After processing, describe your observations in detail for every enhancement step, explaining how the chosen operations improved the visual quality. Finally, write a clear conclusion summarizing the effectiveness and limitations of the applied gray-level techniques. **CO3, CO4**



Code:

```
!pip install -q opencv-python matplotlib numpy
```

```
import cv2  
import numpy as np  
import matplotlib.pyplot as plt
```

```
def show(title, img):  
    plt.figure(figsize=(5,5))  
    plt.imshow(img, cmap='gray')  
    plt.title(title)  
    plt.axis("off")  
    plt.show()
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 4

Date: 28-11-2025

Enrollment No: 92301733054

1. Negative Transform

```
def negative(img):  
    return 255 - img
```

2. Log Transform

```
def log_transform(img):  
    img = img.astype(np.float32)  
    c = 255 / np.log(1 + np.max(img))  
    return np.uint8(c * np.log(1 + img))
```

3. Gamma Correction

```
def gamma_correction(img, gamma):  
    img_norm = img / 255.0  
    corrected = np.power(img_norm, gamma)  
    return np.uint8(corrected * 255)
```

4. Contrast Stretching

```
def contrast_stretch(img):  
    min_val = np.min(img)  
    max_val = np.max(img)  
    stretched = (img - min_val) * (255 / (max_val - min_val + 1e-10))  
    return stretched.astype(np.uint8)
```

5. Histogram Equalization

```
def hist_equalize(img):  
    return cv2.equalizeHist(img)
```

```
image_paths = [  
    "/content/Picture1.png",  
    "/content/Picture2.png",  
    "/content/Picture3.png",  
    "/content/Picture4.png",  
    "/content/Picture5.png"  
]
```

```
print("Enhancing all 5 images...\n")
```

```
for path in image_paths:  
    print("Processing:", path)
```

```
    img = cv2.imread(path, 0)  
    if img is None:  
        print("ERROR: File not found —", path)  
        continue
```

```
    show("Original Image", img)
```


**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 4

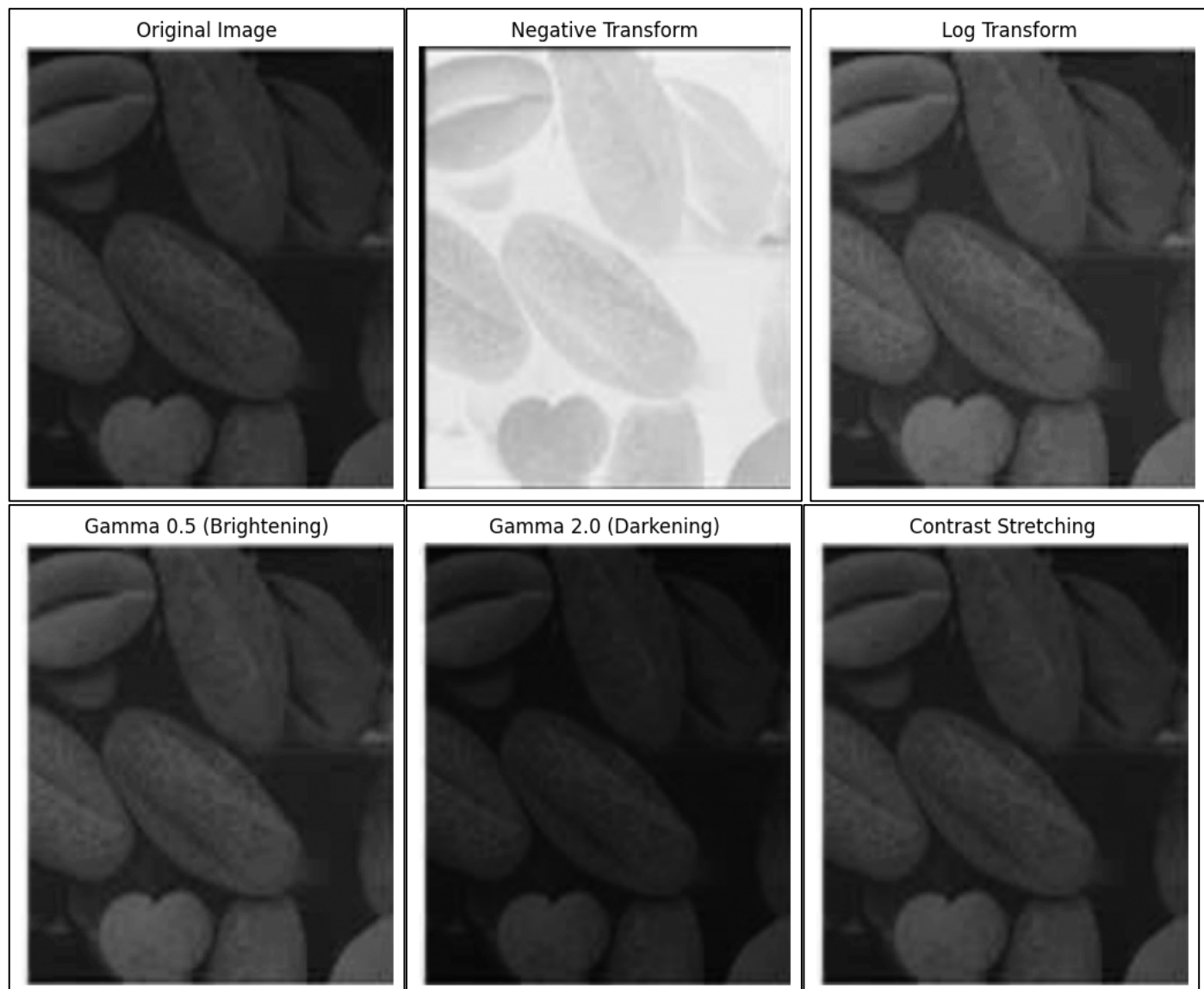
Date: 28-11-2025

Enrollment No: 92301733054

```
show("Negative Transform", negative(img))
show("Log Transform", log_transform(img))
show("Gamma Correction ( $\gamma=0.5$ ) BRIGHT", gamma_correction(img, 0.5))
show("Gamma Correction ( $\gamma=2.0$ ) DARK", gamma_correction(img, 2.0))
show("Contrast Stretching", contrast_stretch(img))
show("Histogram Equalization", hist_equalize(img))
```

```
print("\nDone for:", path, "\n\n")
```

OUTPUT:



**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 4

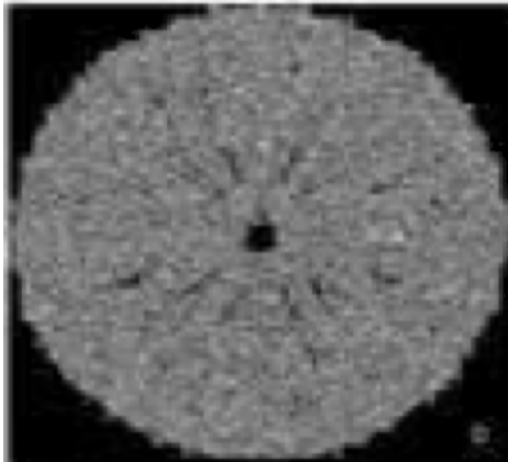
Date: 28-11-2025

Enrollment No: 92301733054

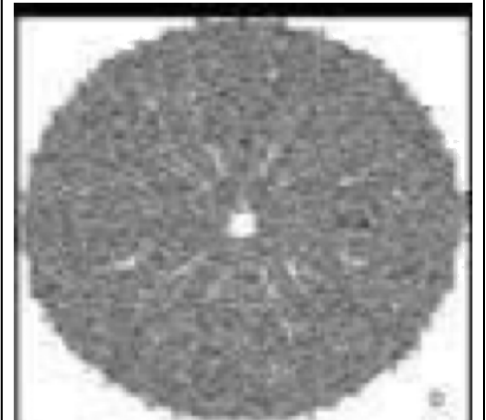
Histogram Equalization



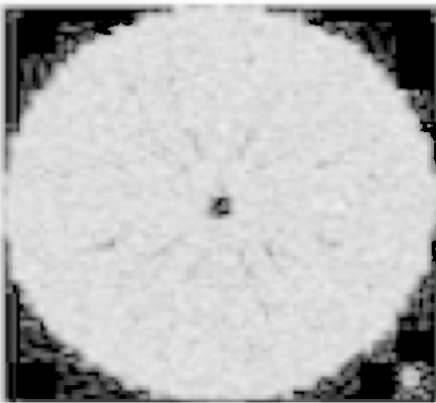
Original Image



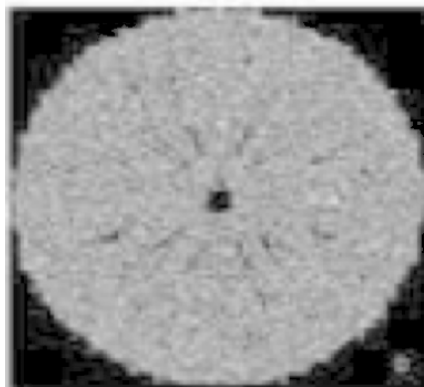
Negative Transform



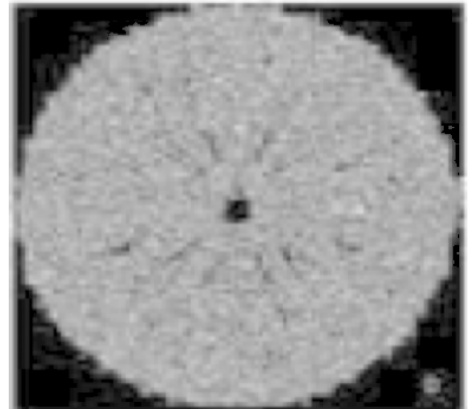
Log Transform



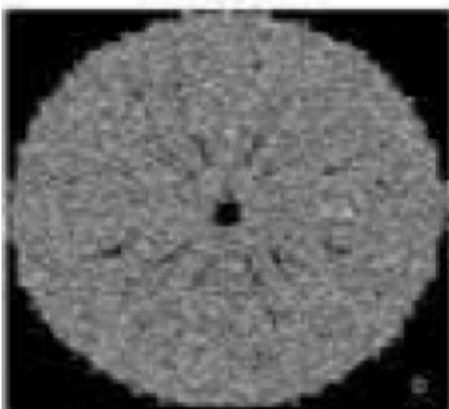
Gamma 0.5 (Brightening)



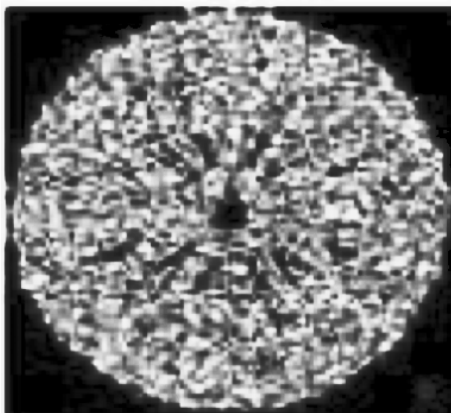
Gamma 0.5 (Brightening)



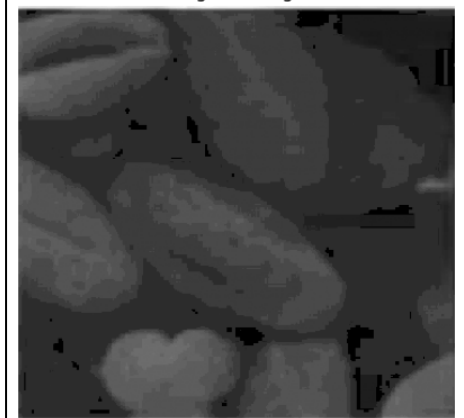
Contrast Stretching



Histogram Equalization



Original Image



**Subject: Digital Signal and
Image Processing(01CT0513)**

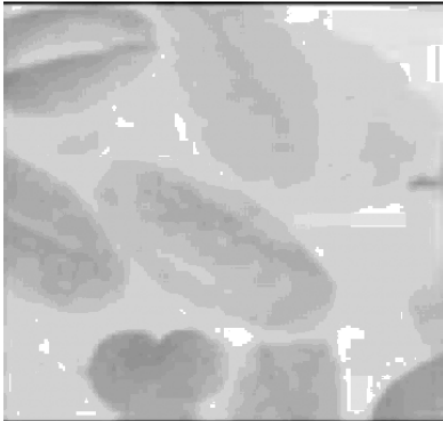
Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 4

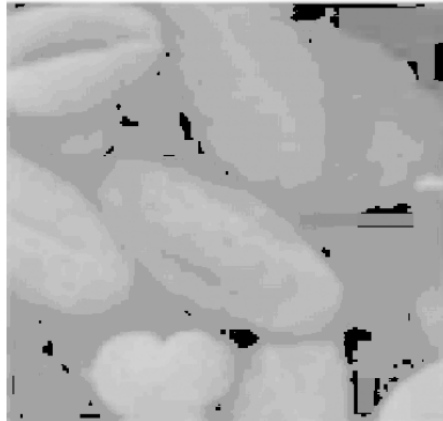
Date: 28-11-2025

Enrollment No: 92301733054

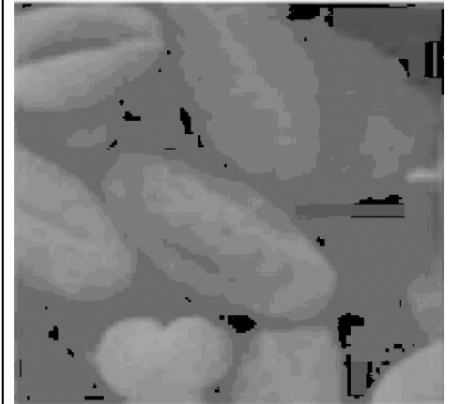
Negative Transform



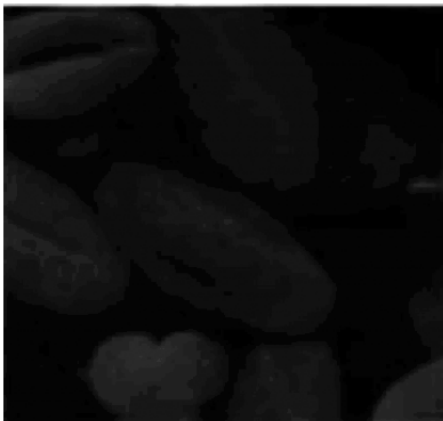
Log Transform



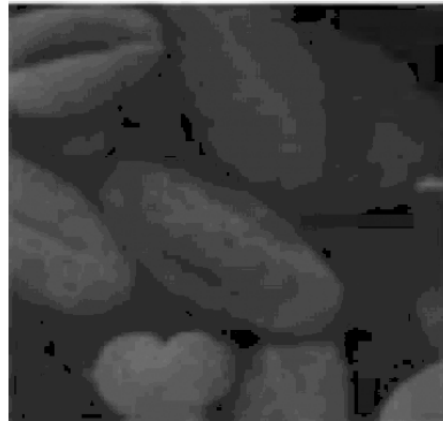
Gamma 0.5 (Brightening)



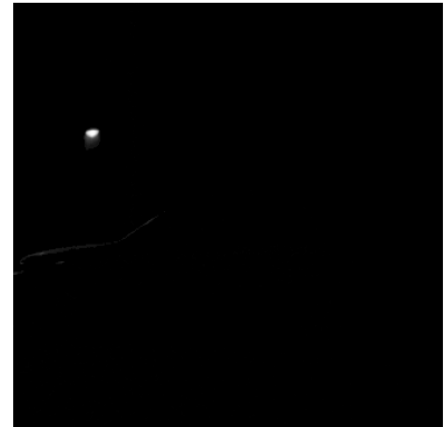
Gamma 2.0 (Darkening)



Contrast Stretching



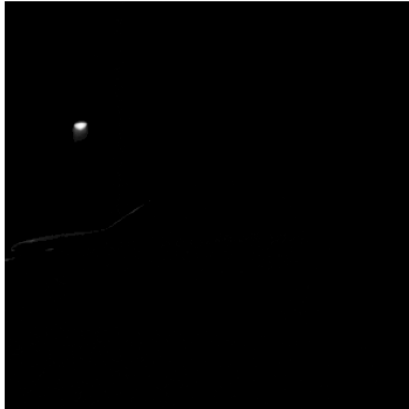
Original Image



Histogram Equalization



Original Image



Negative Transform



**Subject: Digital Signal and
Image Processing(01CT0513)**

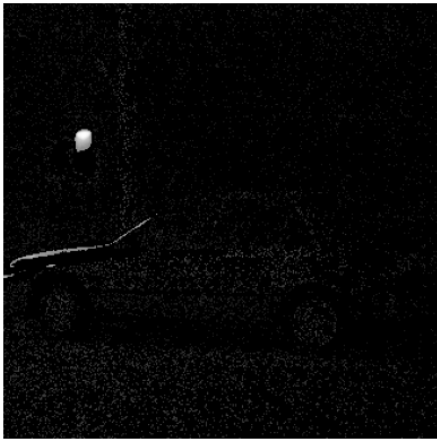
Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 4

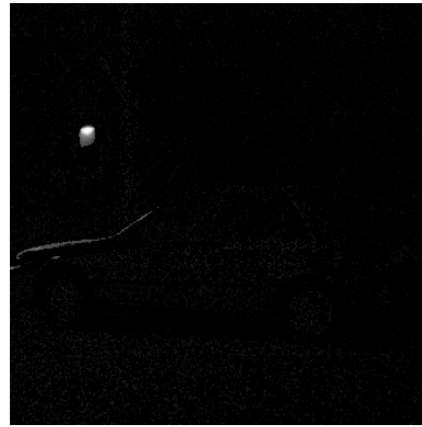
Date: 28-11-2025

Enrollment No: 92301733054

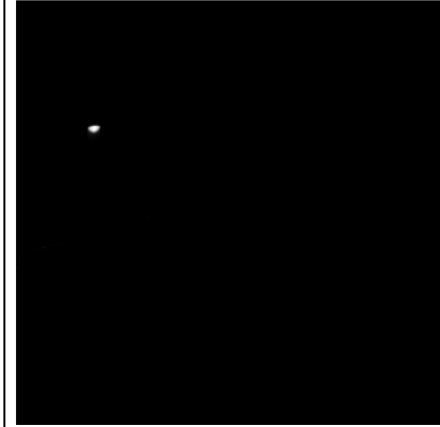
Log Transform



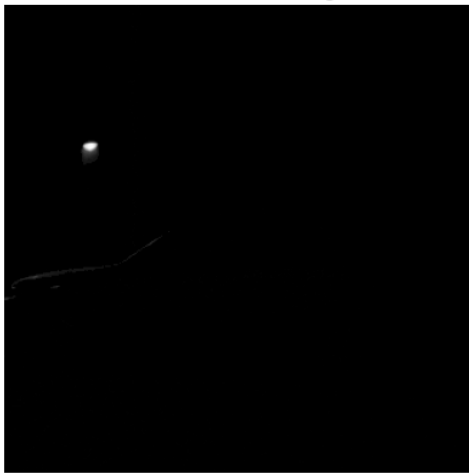
Gamma 0.5 (Brightening)



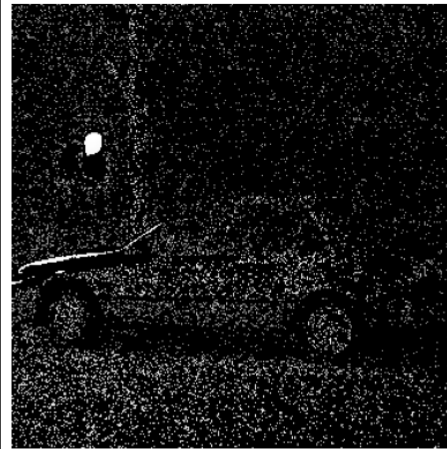
Gamma 2.0 (Darkening)



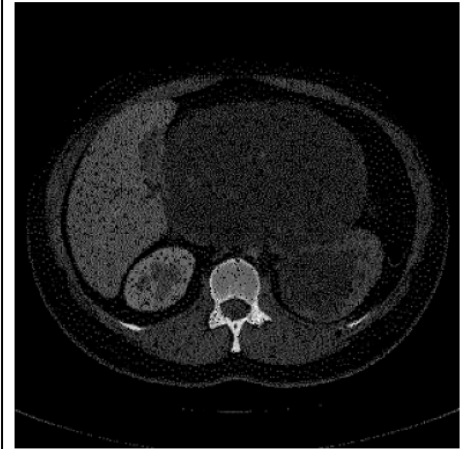
Contrast Stretching



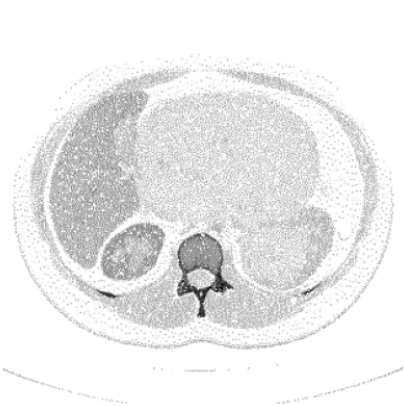
Histogram Equalization



Original Image



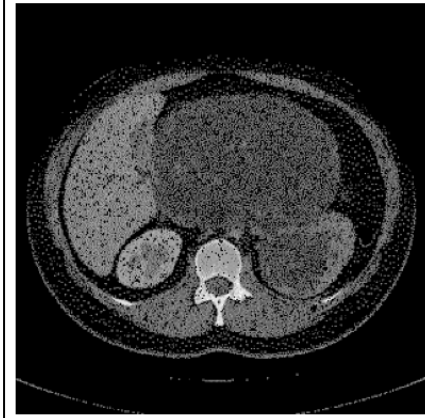
Negative Transform



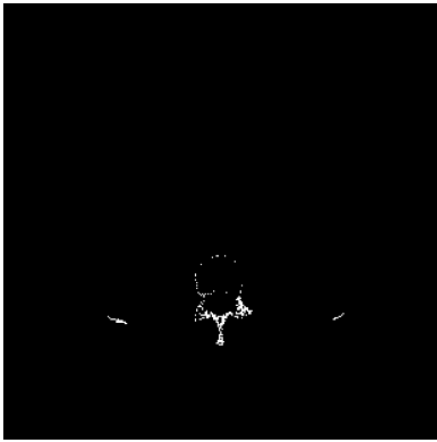
Log Transform



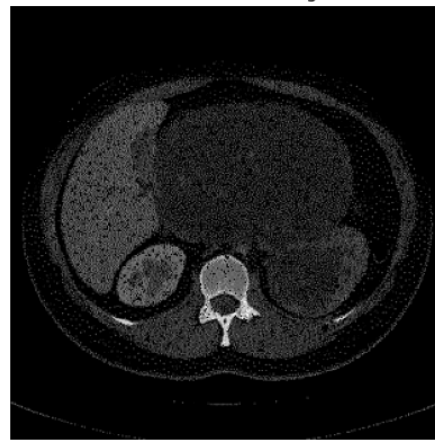
Gamma 0.5 (Brightening)



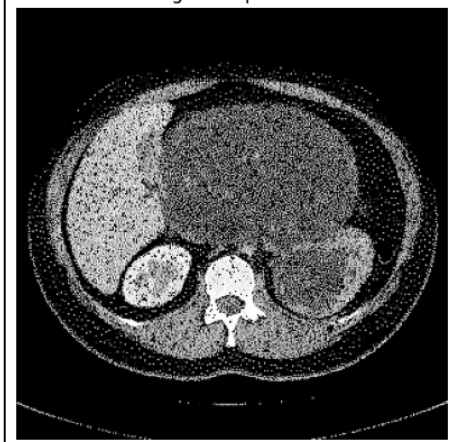
Gamma 2.0 (Darkening)



Contrast Stretching



Histogram Equalization



Obsevation:

1. Negative Transformation

- The negative transform inverted the brightness levels, making dark regions bright and bright regions dark.
- It enhanced details in images where important information was hidden in darker regions.
- However, for images already bright or with low contrast, the negative did not significantly improve visual clarity and sometimes made the image harder to interpret.

2. Log Transformation

- Log transform improved visibility in very dark areas by expanding low-intensity gray levels.
- It compressed the high-intensity regions, reducing over-brightness and enhancing shadow details.
- This was especially useful for images with poor lighting or extremely low contrast.
- However, images that were already well-lit showed minimal benefit.

3. Gamma Correction

$\gamma = 0.5$ (Brightening)

- Mid-tones and shadows became brighter, improving visibility in dark regions.
- Useful for images with underexposure or shadow dominance.
- In some pictures, over-brightening caused loss of detail in already bright areas.

$\gamma = 2.0$ (Darkening)

- Reduced excessive brightness and improved detail retention in highlighted areas.
- Helped control glare and highlight overexposure in light-dominated images.
- In very dark images, it made visibility worse.

 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN HANDED QUESTIONS	
QUESTION 4	Date: 28-11-2025	Enrollment No: 92301733054

4. Contrast Stretching

- Produced one of the most visually effective enhancements across all images.
- Increased separation between dark and bright pixels by stretching pixel values across the full intensity range [0–255].
- Significantly improved details in low-contrast images.
- Works best when the image histogram is narrow (poor contrast).
- In noisy images, it sometimes amplified noise along with useful details.

5. Histogram Equalization

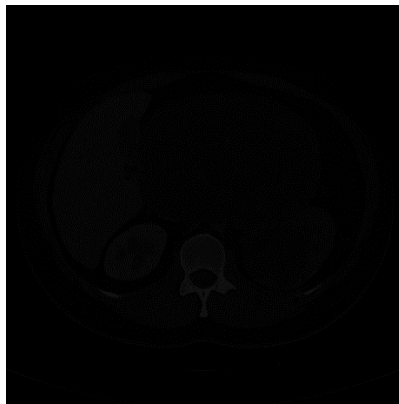
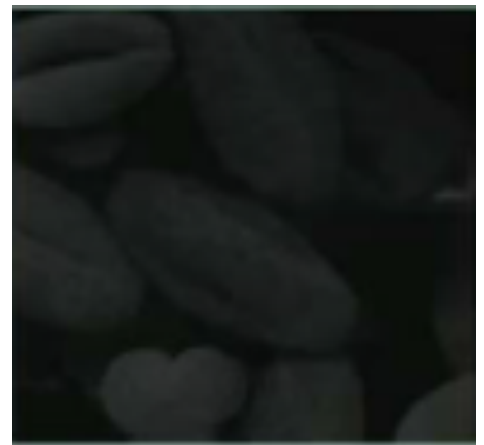
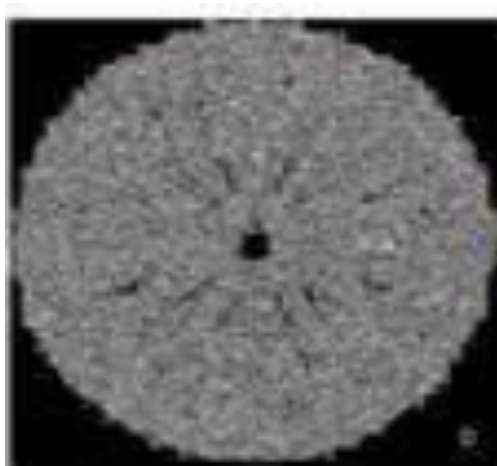
- Provided strong global contrast enhancement and improved fine details that were previously hidden.
- Worked especially well in images with uneven or faded lighting.
- Improved clarity of textures and edges.
- However, for images with already good contrast, it introduced artifacts or unnatural brightness jumps due to over-equalization.
- In some smooth regions, noise became more visible.

Conclusion:

we observe that gray-level transformation techniques significantly improve degraded images, but their effectiveness varies depending on the type of degradation. Log and gamma corrections help enhance dark or overly bright regions by redistributing pixel intensity values. Contrast stretching and histogram equalization provide strong overall contrast enhancement, revealing hidden details and improving visual clarity. However, these techniques also have limitations—over-enhancement can introduce artifacts, amplify noise, or distort natural brightness levels. Overall, gray-level transformations are powerful tools for basic image enhancement, but selecting the appropriate method depends on the specific characteristics of each degraded image.

 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN HANDED QUESTIONS	
QUESTION 6	Date: 28-11-2025	Enrollment No: 92301733054

Q6. Using the same set of degraded images provided in the OPE5, apply histogram equalization and histogram matching techniques to enhance their visual quality. Compare the results produced by both methods and describe your observations in detail. Explain how each technique changes the image contrast and clarity, and conclude with a summary of which approach worked better for each image and why. **CO3, CO4**



Code:

```
!pip install -q opencv-python matplotlib numpy scikit-image
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import exposure
```

```
def show_side_by_side(title, img1, img2, lab1="Method 1", lab2="Method 2"):
    plt.figure(figsize=(10,4))
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 6

Date: 28-11-2025

Enrollment No: 92301733054

```
plt.subplot(1,2,1)
plt.imshow(img1, cmap='gray')
plt.title(lab1)
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(img2, cmap='gray')
plt.title(lab2)
plt.axis("off")

plt.suptitle(title, fontsize=14)
plt.show()

image_paths = [
    "/content/Picture1.png",
    "/content/Picture2.png",
    "/content/Picture3.png",
    "/content/Picture4.png",
    "/content/Picture5.png"
]

reference_img = cv2.imread(image_paths[0], 0)

print("Using Picture1 as histogram reference.\n")

for path in image_paths:
    print("Processing:", path)

    img = cv2.imread(path, 0)

    if img is None:
        print("ERROR: File not found:", path)
        continue

    heq = cv2.equalizeHist(img)

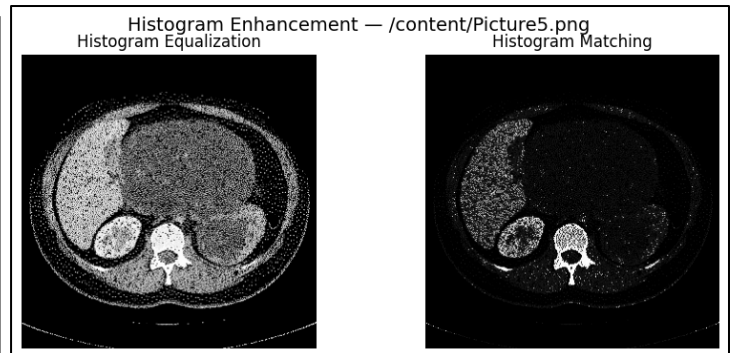
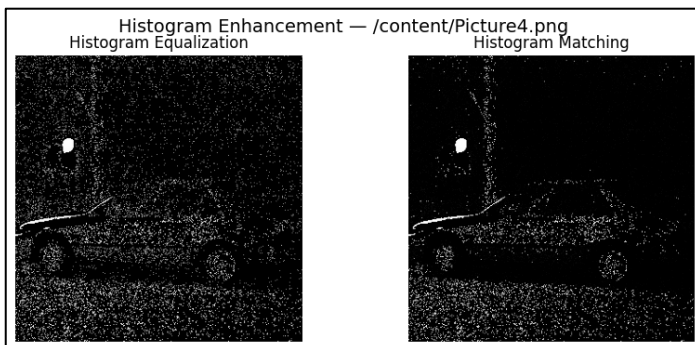
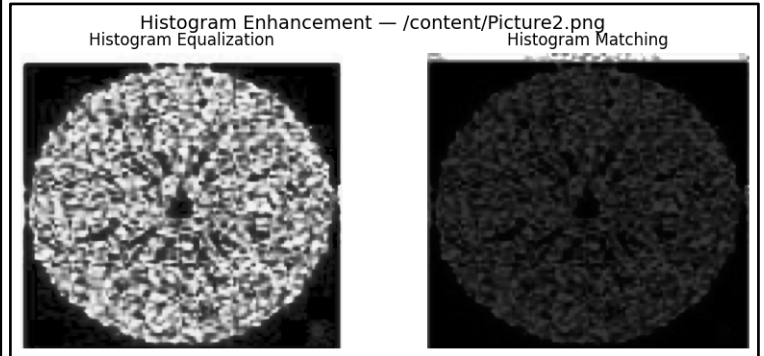
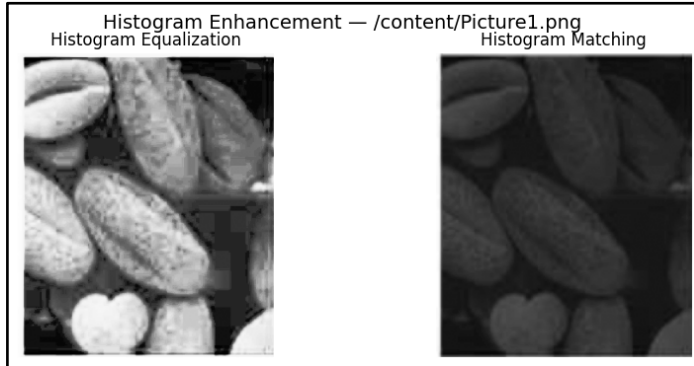
    img_float = img.astype(np.float32) / 255.0
    ref_float = reference_img.astype(np.float32) / 255.0

    hmatch = exposure.match_histograms(img_float, ref_float)
    hmatch = np.uint8(hmatch * 255)

    show_side_by_side(
        title=f"Histogram Enhancement — {path}",
        img1=heq,
        img2=hmatch,
        lab1="Histogram Equalization",
```

)
print("Done.\n\n")

OUTPUT:



Obsevation:

1. Histogram Equalization Results

- Histogram equalization noticeably improved the overall contrast of all degraded images by stretching and redistributing pixel intensities across the full dynamic range.
- Dark regions became brighter, and hidden details in low-light images became more visible.
- Images with faded or washed-out brightness showed stronger contrast and better definition of edges and textures.
- However, histogram equalization sometimes introduced **over-enhancement**, leading to:
 - unnatural brightness jumps,
 - amplified noise in smooth areas,
 - graininess in low-contrast regions.

This is because equalization globally modifies the entire histogram and does not consider the original visual balance of the image.



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN HANDED QUESTIONS

QUESTION 6

Date: 28-11-2025

Enrollment No: 92301733054

2. Histogram Matching Results

- Histogram matching produced smoother and more natural-looking images when matched with the reference image's intensity distribution (Picture1).
- Enhancement was more controlled, without over-sharpening or artificial contrast stretching.
- Dark and bright areas were balanced in a way that preserved the reference image's tonal style.
- Images with unusual lighting improved in clarity without exaggerating noise.
- Unlike histogram equalization, the brightness transitions remained smooth and visually pleasing.

However, histogram matching depends heavily on the **quality and style of the reference image**. If the reference image has different lighting or content, the output may not be optimal.

3. Comparison Between Both Methods

- **Histogram Equalization** provided **stronger contrast enhancement** and revealed details aggressively, but sometimes caused noise amplification and unnatural brightness levels.
- **Histogram Matching** provided **more natural enhancement**, maintaining smoother gradients while improving clarity based on the reference image's tone distribution.
- For images with heavy degradation or extreme low contrast, histogram equalization showed more immediate improvement.
- For images requiring subtle and balanced enhancement, histogram matching performed better.

Conclusion:

In this question, both histogram equalization and histogram matching successfully enhanced the visual quality of the degraded images, but each method worked differently. Histogram equalization provided strong contrast improvement by spreading pixel intensities across the full range, making hidden details more visible; however, it sometimes introduced noise and produced unnatural brightness variations. Histogram matching delivered more balanced and natural-looking results by shaping the image to follow the reference histogram, maintaining smoother tonal transitions and avoiding excessive enhancement. Overall, histogram equalization is more effective for severely low-contrast images, while histogram matching is preferable when a controlled and visually consistent enhancement is required.

 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN ENDED QUESTIONS	
QUESTION 7	Date: 28-11-2025	Enrollment No: 92301733054

Q7. Use appropriate smoothing and sharpening techniques to enhance the clarity of the images provided in the Module Section under Experiment 9. **CO3, CO4**



Apply multiple filters, observe the changes produced by each method, and document your observations in detail. Discuss how smoothing and sharpening individually affect image quality, and summarize your conclusions based on the results.



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 7

Date: 28-11-2025

Enrollment No: 92301733054

Code:

```
!pip install -q opencv-python matplotlib numpy
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
image_paths = [
    "/content/Picture1.png",
    "/content/Picture2.png",
    "/content/Picture3.png",
    "/content/Picture4.png",
    "/content/Picture5.png"
]
```

```
images = []
```

```
for p in image_paths:
    img = cv2.imread(p, 0)
    if img is not None:
        images.append(img)
        print("Loaded:", p)
    else:
        print("Missing:", p)
```

```
smooth_results = []
sharp_results = []
```

```
for img in images:
```

```
    avg = cv2.blur(img, (5,5))
    gauss = cv2.GaussianBlur(img, (5,5), 1)
    median = cv2.medianBlur(img, 5)
    bilateral = cv2.bilateralFilter(img, 9, 75, 75)
```

```
    smooth_results.append([img, avg, gauss, median, bilateral])
```

```
    lap = cv2.Laplacian(img, cv2.CV_64F)
    lap = cv2.convertScaleAbs(lap)
    lap_sharp = cv2.addWeighted(img, 1.0, lap, 1.0, 0)
```

```
    blur = cv2.GaussianBlur(img, (9,9), 2)
```




Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 7

Date: 28-11-2025

Enrollment No: 92301733054

```
unsharp = cv2.addWeighted(img, 1.5, blur, -0.5, 0)
```

```
k = 1.8
```

```
highboost = cv2.addWeighted(img, k, blur, -(k-1), 0)
```

```
sharp_results.append([img, lap_sharp, unsharp, highboost])
```

```
titles_smooth = ["Original", "Averaging", "Gaussian", "Median", "Bilateral"]
```

```
plt.figure(figsize=(14, 12))
```

```
plt.suptitle("Smoothing Results for All Images", fontsize=18)
```

```
rows = len(images)
```

```
cols = 5
```

```
for r in range(rows):
```

```
    for c in range(cols):
```

```
        plt.subplot(rows, cols, r*cols + c + 1)
```

```
        plt.imshow(smooth_results[r][c], cmap='gray')
```

```
        if r == 0:
```

```
            plt.title(titles_smooth[c], fontsize=12)
```

```
        plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
titles_sharp = ["Original", "Laplacian", "Unsharp Mask", "High-Boost"]
```

```
plt.figure(figsize=(14, 10))
```

```
plt.suptitle("Sharpening Results for All Images", fontsize=18)
```

```
rows = len(images)
```

```
cols = 4
```

```
for r in range(rows):
```

```
    for c in range(cols):
```

```
        plt.subplot(rows, cols, r*cols + c + 1)
```

```
        plt.imshow(sharp_results[r][c], cmap='gray')
```

```
        if r == 0:
```

```
            plt.title(titles_sharp[c], fontsize=12)
```

```
        plt.axis("off")
```

```
plt.tight_layout()
```

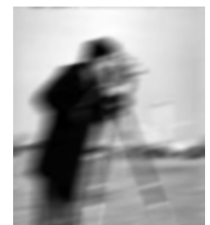
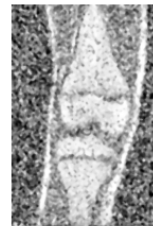
```
plt.show()
```

**Subject: Digital Signal and
Image Processing(01CT0513)**
Aim: FINAL OPEN ENDED QUESTIONS
QUESTION 7
Date: 28-11-2025
Enrollment No: 92301733054
OUTPUT:
Smoothing Results for All Images
Original

Averaging

Gaussian

Median

Bilateral


 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN ENDED QUESTIONS	
QUESTION 7	Date: 28-11-2025	Enrollment No: 92301733054

Obsevation:

Smoothing Techniques

- **Averaging Filter:** Reduced noise but caused noticeable blurring and loss of detail.
- **Gaussian Filter:** Smoothed images more naturally and preserved edges better than averaging.
- **Median Filter:** Very effective for salt-and-pepper noise; preserved edges well.
- **Bilateral Filter:** Best smoothing output; removed noise while keeping edges sharp and clear.

Sharpening Techniques

- **Laplacian Sharpening:** Enhanced edges strongly but amplified noise in degraded images.
- **Unsharp Masking:** Produced clean and natural sharpening with good detail recovery.
- **High-Boost Filter:** Provided strongest sharpening but increased noise if boosted too much.

Conclusion:

Smoothing and sharpening techniques each improve image quality in different ways. Smoothing filters reduce noise and make images cleaner, with the bilateral and median filters performing best by preserving important edges. Sharpening filters enhance details and improve clarity, with unsharp masking giving the most natural results. Overall, a combination of smoothing (to remove noise) followed by sharpening (to restore fine details) provides the best enhancement for degraded images.

 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN ENDED QUESTIONS	
QUESTION 7	Date: 28-11-2025	Enrollment No: 92301733054

Q8. Apply non-linear filters (median or other rank filters) to remove noise from data/images provided in OPE7 and perform comprehensive analysis. **CO3, CO4**

Identify noise types present in the dataset (salt-and-pepper, impulse, speckle, etc.). Implement median filtering with varying window sizes and, optionally, other non-linear filters (alpha-trimmed mean, morphological filters). Analyze how filter size and type affect detail preservation vs. noise removal. Observations: optimum filter sizes per noise type, edge preservation analysis, and failure cases. Summarize best-performing methods for each noise type and practical recommendations.

Code:

```
!pip install -q scikit-image opencv-python matplotlib numpy scipy pandas
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import util, filters, morphology, exposure
from skimage.metrics import peak_signal_noise_ratio as PSNR, structural_similarity as SSIM
from scipy.ndimage import median_filter
import pandas as pd
from skimage.feature import canny
```

```
def show(title, img1, img2, l1="Original", l2="Filtered"):
    plt.figure(figsize=(10,4))
    plt.suptitle(title)
    plt.subplot(1,2,1); plt.imshow(img1, cmap='gray'); plt.title(l1); plt.axis("off")
    plt.subplot(1,2,2); plt.imshow(img2, cmap='gray'); plt.title(l2); plt.axis("off")
    plt.show()
```

```
def to_float(img):
    if img.dtype == np.uint8:
        return img.astype(np.float32)/255.0
    return img.astype(np.float32)
```

```
def to_uint8(img):
    return np.clip(img*255.0,0,255).astype(np.uint8)
```

```
# Noise addition
def add_salt_pepper(img, amount=0.05):
    return util.random_noise(img, mode='s&p', amount=amount)
```

```
def add_speckle(img, var=0.04):
    return util.random_noise(img, mode='speckle', var=var)
```



Marwadi
University

Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 7

Date: 28-11-2025

Enrollment No: 92301733054

```
def add_gaussian(img, var=0.004):  
    return util.random_noise(img, mode='gaussian', var=var)
```

```
def alpha_trimmed_mean(img, size=3, d=2):  
    pad = size//2  
    padded = np.pad(img, pad, mode='reflect')  
    out = np.zeros_like(img)  
    flat_n = size*size  
    trim = d//2  
  
    for i in range(out.shape[0]):  
        for j in range(out.shape[1]):  
            block = padded[i:i+size, j:j+size].flatten()  
            block_sorted = np.sort(block)  
            trimmed = block_sorted[trim:flat_n-trim]  
            out[i,j] = np.mean(trimmed)  
    return out
```

```
def edge_overlap(ref, proc):  
    ref_e = canny(ref)  
    proc_e = canny(proc)  
    inter = np.logical_and(ref_e, proc_e).sum()  
    return inter / max(ref_e.sum(),1)
```

```
image_paths = [  
    "/content/Picture1.png",  
    "/content/Picture2.png",  
    "/content/Picture3.png",  
    "/content/Picture4.png",  
    "/content/Picture5.png"  
]
```

```
images = {}  
for p in image_paths:  
    img = cv2.imread(p,0)  
    if img is not None:  
        images[p] = to_float(img)  
        print("Loaded:", p)  
    else:  
        print(" Missing:", p)
```

```
if not images:  
    raise RuntimeError("No images loaded.")
```

```
noisy_versions = {}
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 7

Date: 28-11-2025

Enrollment No: 92301733054

for p, img in images.items():

```
noisy_versions[p] = {  
    "salt_pepper": add_salt_pepper(img, amount=0.06),  
    "speckle": add_speckle(img, var=0.03),  
    "gaussian": add_gaussian(img, var=0.005)  
}  
print("Created noise versions for:", p)
```

```
median_sizes = [3,5,7,9]  
alpha_sizes = [3,5]  
alpha_d = 2  
morph_sizes = [3,5,7]
```

```
rows = []
```

```
for p, noises in noisy_versions.items():  
    orig = images[p]
```

```
    for noise_type, noisy in noises.items():  
        print(f"Image: {p} | Noise: {noise_type}")
```

```
        # baseline  
        base_ps = PSNR(orig, noisy, data_range=1)  
        base_ss = SSIM(orig, noisy, data_range=1)  
        base_eo = edge_overlap(orig, noisy)  
        print("Baseline:", base_ps, base_ss, base_eo)
```

```
    for k in median_sizes:  
        med = median_filter(noisy, size=k)  
        ps = PSNR(orig, med, data_range=1)  
        ss = SSIM(orig, med, data_range=1)  
        eo = edge_overlap(orig, med)  
        rows.append([p, noise_type, f"median_{k}", ps, ss, eo])  
        print(f"Median {k}: PSNR={ps:.2f} SSIM={ss:.4f} Edge={eo:.3f}")
```

```
    for k in alpha_sizes:  
        at = alpha_trimmed_mean((noisy*255).astype(np.uint8), size=k, d=alpha_d)/255.0  
        ps = PSNR(orig, at, data_range=1)  
        ss = SSIM(orig, at, data_range=1)  
        eo = edge_overlap(orig, at)  
        rows.append([p, noise_type, f"alpha_trim_{k}", ps, ss, eo])  
        print(f"Alpha-trim {k}: PSNR={ps:.2f} SSIM={ss:.4f} Edge={eo:.3f}")
```

```
    for s in morph_sizes:  
        se = morphology.square(s)
```




Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 7

Date: 28-11-2025

Enrollment No: 92301733054

```
op = morphology.opening((noisy*255).astype(np.uint8), se)  
cl = morphology.closing(op, se).astype(np.float32)/255.0
```

```
ps = PSNR(orig, cl, data_range=1)  
ss = SSIM(orig, cl, data_range=1)  
eo = edge_overlap(orig, cl)  
rows.append([p, noise_type, f'morph_{s}', ps, ss, eo])  
print(f'Morph {s}: PSNR={ps:.2f} SSIM={ss:.4f} Edge={eo:.3f}')
```

Show best median visually

best = None

for k in median_sizes:

med = median_filter(noisy, size=k)

ss = SSIM(orig, med, data_range=1)

if best is None or ss > best[0]:

best = (ss, k, med)

```
show(f'{p} | {noise_type}: Best Median {best[1]} (SSIM={best[0]:.4f})',  
to_uint8(orig), to_uint8(best[2]),  
"Clean", "Best Median")
```

```
df = pd.DataFrame(rows, columns=["image", "noise", "method", "psnr", "ssim", "edge_overlap"])  
summary = df.sort_values(["image", "noise", "ssim", "psnr"], ascending=[True, True, False, False])  
print("Top performers:")  
display(summary.head(25))
```

print("Practical Recommendations:")

print("1) Salt-and-pepper noise → Median 3x3 or 5x5, Morphological opening+closing")

print("2) Impulse noise → Median 5x5 or Alpha-trim filter (size 5)")

print("3) Speckle noise → Alpha-trim, median 7x7; morphological less effective")

print("4) Gaussian noise → Non-linear filters less effective; small median helps but may blur edges")

print("5) Edge preservation best → Median small window (3 or 5)")

print("6) Larger window sizes remove more noise but reduce fine detail.")

OUTPUT:

Baseline PSNR: 16.83, SSIM: 0.4157, EdgeOverlap: 0.6161

median 3: PSNR=27.32, SSIM=0.8325, EdgeOv=0.6396

median 5: PSNR=23.80, SSIM=0.6127, EdgeOv=0.2810

median 7: PSNR=22.63, SSIM=0.4818, EdgeOv=0.1347

median 9: PSNR=22.17, SSIM=0.4240, EdgeOv=0.0839

alpha-trim 3: PSNR=25.94, SSIM=0.7781, EdgeOv=0.6355

alpha-trim 5: PSNR=23.65, SSIM=0.5775, EdgeOv=0.2676

alpha-trim 7: PSNR=22.62, SSIM=0.4494, EdgeOv=0.0963

morph 3: PSNR=22.92, SSIM=0.6404, EdgeOv=0.3098

morph 5: PSNR=19.31, SSIM=0.3946, EdgeOv=0.0948

morph 7: PSNR=14.95, SSIM=0.2768, EdgeOv=0.0753

**Subject: Digital Signal and
Image Processing(01CT0513)**
Aim: FINAL OPEN ENDED QUESTIONS
QUESTION 7
Date: 28-11-2025
Enrollment No: 92301733054

/tmp/ipython-input-4145038815.py:48: FutureWarning: `square` is deprecated since version 0.25 and will be removed in version 0.27. Use `skimage.morphology.footprint\_rectangle` instead.

```
selem = morphology.square(s)
```

Best median size (by SSIM) = 3 (SSIM=0.8325)

/content/Picture1.png | s&p : Original vs BestMedian3
Original Median 3



Baseline PSNR: 21.54, SSIM: 0.6297, EdgeOverlap: 0.7454

median 3: PSNR=25.08, SSIM=0.6613, EdgeOv=0.5833

median 5: PSNR=23.33, SSIM=0.5339, EdgeOv=0.2604

median 7: PSNR=22.43, SSIM=0.4421, EdgeOv=0.1088

median 9: PSNR=22.04, SSIM=0.3997, EdgeOv=0.0621

alpha-trim 3: PSNR=25.50, SSIM=0.7050, EdgeOv=0.6235

alpha-trim 5: PSNR=23.79, SSIM=0.5720, EdgeOv=0.2645

alpha-trim 7: PSNR=22.81, SSIM=0.4604, EdgeOv=0.0787

morph 3: PSNR=20.05, SSIM=0.5220, EdgeOv=0.3065

morph 5: PSNR=16.78, SSIM=0.3504, EdgeOv=0.0821

morph 7: PSNR=15.47, SSIM=0.3158, EdgeOv=0.0625

Best median size (by SSIM) = 3 (SSIM=0.6613)

/content/Picture1.png | speckle : Original vs BestMedian3
Original Median 3



**Subject: Digital Signal and
Image Processing(01CT0513)**
Aim: FINAL OPEN ENDED QUESTIONS
QUESTION 7
Date: 28-11-2025
Enrollment No: 92301733054

Baseline PSNR: 23.38, SSIM: 0.6221, EdgeOverlap: 0.7071

median 3: PSNR=25.93, SSIM=0.7217, EdgeOv=0.5836

median 5: PSNR=23.75, SSIM=0.5771,

EdgeOv=0.2686

median 7: PSNR=22.78, SSIM=0.4657,

EdgeOv=0.1087

median 9: PSNR=22.36, SSIM=0.4114,

EdgeOv=0.0634

alpha-trim 3: PSNR=26.04, SSIM=0.7568,

EdgeOv=0.6046

alpha-trim 5: PSNR=23.92, SSIM=0.5950,

EdgeOv=0.2458

alpha-trim 7: PSNR=22.87, SSIM=0.4690,

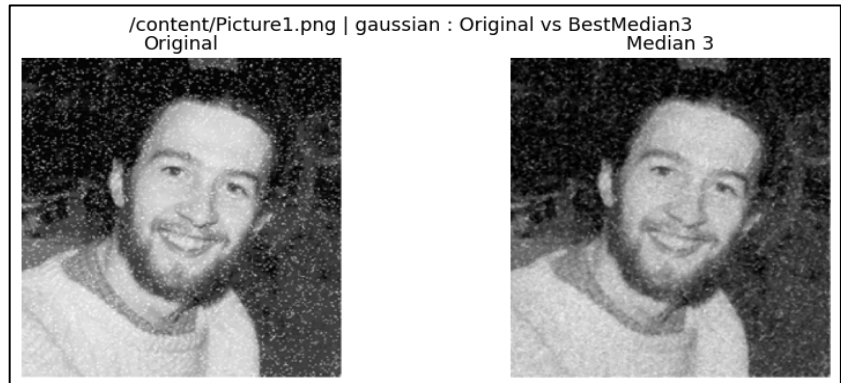
EdgeOv=0.0769

morph 3: PSNR=21.27, SSIM=0.5550, EdgeOv=0.3132

morph 5: PSNR=18.11, SSIM=0.3709, EdgeOv=0.0922

morph 7: PSNR=16.97, SSIM=0.3290, EdgeOv=0.0701

Best median size (by SSIM) = 3 (SSIM=0.7217)



Baseline PSNR: 16.89, SSIM: 0.7391, EdgeOverlap: 0.7645

median 3: PSNR=21.69, SSIM=0.8715, EdgeOv=0.7507

median 5: PSNR=17.62, SSIM=0.6108,

EdgeOv=0.4451

median 7: PSNR=15.51, SSIM=0.3374,

EdgeOv=0.2340

median 9: PSNR=14.71, SSIM=0.2001,

EdgeOv=0.1769

alpha-trim 3: PSNR=21.16,

SSIM=0.8479, EdgeOv=0.7420

alpha-trim 5: PSNR=17.32,

SSIM=0.5706, EdgeOv=0.4018

alpha-trim 7: PSNR=15.42,

SSIM=0.3047, EdgeOv=0.1913

morph 3: PSNR=17.29, SSIM=0.6838,

EdgeOv=0.4880

morph 5: PSNR=11.54, SSIM=0.2981,

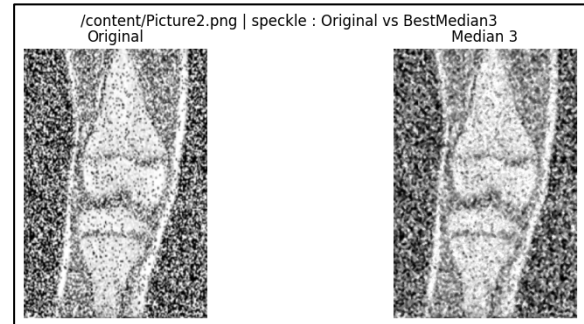
EdgeOv=0.2180

morph 7: PSNR=8.29, SSIM=0.1327, EdgeOv=0.1465

Best median size (by SSIM) = 3 (SSIM=0.8715)



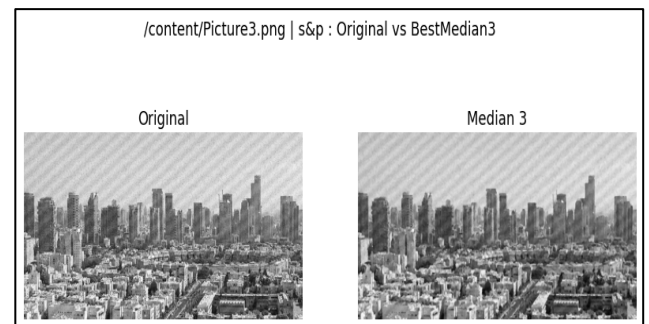
Baseline PSNR: 19.83, SSIM: 0.8051, EdgeOverlap: 0.7957
 median 3: PSNR=20.59, SSIM=0.8125, EdgeOv=0.6877
 median 5: PSNR=17.43, SSIM=0.5843, EdgeOv=0.4152
 median 7: PSNR=15.48, SSIM=0.3236, EdgeOv=0.2263
 median 9: PSNR=14.72, SSIM=0.1901, EdgeOv=0.1760
 alpha-trim 3: PSNR=20.84, SSIM=0.8228, EdgeOv=0.7136
 alpha-trim 5: PSNR=17.29, SSIM=0.5610, EdgeOv=0.4023
 alpha-trim 7: PSNR=15.40, SSIM=0.3001, EdgeOv=0.1858
 morph 3: PSNR=16.05, SSIM=0.5873, EdgeOv=0.4431
 morph 5: PSNR=11.83, SSIM=0.2722, EdgeOv=0.2089
 morph 7: PSNR=9.92, SSIM=0.1654, EdgeOv=0.1410
 Best median size (by SSIM) = 3 (SSIM=0.8125)



Baseline PSNR: 23.39, SSIM: 0.9075, EdgeOverlap: 0.8494
 median 3: PSNR=21.41, SSIM=0.8516, EdgeOv=0.7309
 median 5: PSNR=17.63, SSIM=0.6017, EdgeOv=0.4398
 median 7: PSNR=15.54, SSIM=0.3321, EdgeOv=0.2310
 median 9: PSNR=14.75, SSIM=0.1963, EdgeOv=0.1732
 alpha-trim 3: PSNR=21.50, SSIM=0.8557, EdgeOv=0.7523
 alpha-trim 5: PSNR=17.47, SSIM=0.5815, EdgeOv=0.4133
 alpha-trim 7: PSNR=15.50, SSIM=0.3118, EdgeOv=0.1847
 morph 3: PSNR=17.75, SSIM=0.6707, EdgeOv=0.4846
 morph 5: PSNR=12.88, SSIM=0.3299, EdgeOv=0.2274
 morph 7: PSNR=10.61, SSIM=0.2091, EdgeOv=0.1553
 Best median size (by SSIM) = 3 (SSIM=0.8516)



Baseline PSNR: 17.35, SSIM: 0.5159, EdgeOverlap: 0.6488
 median 3: PSNR=23.96, SSIM=0.7646, EdgeOv=0.5596
 median 5: PSNR=21.10, SSIM=0.5711, EdgeOv=0.2796
 median 7: PSNR=19.99, SSIM=0.4703, EdgeOv=0.1732
 median 9: PSNR=19.37, SSIM=0.4100, EdgeOv=0.1311
 alpha-trim 3: PSNR=23.27, SSIM=0.7138, EdgeOv=0.5731
 alpha-trim 5: PSNR=20.55, SSIM=0.5101, EdgeOv=0.2469
 alpha-trim 7: PSNR=19.56, SSIM=0.4146, EdgeOv=0.1098
 morph 3: PSNR=20.21, SSIM=0.6335, EdgeOv=0.3601
 morph 5: PSNR=15.74, SSIM=0.4274, EdgeOv=0.1954
 morph 7: PSNR=11.55, SSIM=0.2685, EdgeOv=0.1343
 Best median size (by SSIM) = 3 (SSIM=0.7646)



Baseline PSNR: 18.02, SSIM: 0.2821, EdgeOverlap: 0.5201
 median 3: PSNR=29.31, SSIM=0.6128, EdgeOv=0.5653
 median 5: PSNR=29.15, SSIM=0.5952, EdgeOv=0.2809
 median 7: PSNR=29.30, SSIM=0.6267, EdgeOv=0.1355
 median 9: PSNR=28.19, SSIM=0.5194, EdgeOv=0.0779
 alpha-trim 3: PSNR=28.14, SSIM=0.5645, EdgeOv=0.5557
 alpha-trim 5: PSNR=28.17, SSIM=0.5501, EdgeOv=0.2165
 alpha-trim 7: PSNR=28.31, SSIM=0.5826, EdgeOv=0.0571
 morph 3: PSNR=26.77, SSIM=0.6550, EdgeOv=0.3949
 morph 5: PSNR=22.91, SSIM=0.5582, EdgeOv=0.1993
 morph 7: PSNR=16.81, SSIM=0.4338, EdgeOv=0.1350
 Best median size (by SSIM) = 7 (SSIM=0.6267)

/content/Picture4.png | s&p : Original vs BestMedian7



Baseline PSNR: 17.50, SSIM: 0.1833, EdgeOverlap: 0.4142
 median 3: PSNR=47.77, SSIM=0.9943, EdgeOv=0.7291
 median 5: PSNR=45.28, SSIM=0.9883, EdgeOv=0.5753
 median 7: PSNR=42.05, SSIM=0.9769, EdgeOv=0.4032
 median 9: PSNR=39.20, SSIM=0.9621, EdgeOv=0.2418
 alpha-trim 3: PSNR=33.22, SSIM=0.8327, EdgeOv=0.5451
 alpha-trim 5: PSNR=33.91, SSIM=0.8445, EdgeOv=0.2915
 alpha-trim 7: PSNR=33.68, SSIM=0.8638, EdgeOv=0.1239
 morph 3: PSNR=34.86, SSIM=0.9791, EdgeOv=0.5604
 morph 5: PSNR=26.37, SSIM=0.9116, EdgeOv=0.2785
 morph 7: PSNR=16.88, SSIM=0.7288, EdgeOv=0.1622
 Best median size (by SSIM) = 3 (SSIM=0.9943)

/content/Picture5.png | s&p : Original vs BestMedian3
 Original Median 3



Baseline PSNR: 23.16, SSIM: 0.2552, EdgeOverlap: 0.4108
 median 3: PSNR=30.62, SSIM=0.6464, EdgeOv=0.3674
 median 5: PSNR=34.45, SSIM=0.8355, EdgeOv=0.2858
 median 7: PSNR=35.88, SSIM=0.8971, EdgeOv=0.1800
 median 9: PSNR=35.84, SSIM=0.9129, EdgeOv=0.1151
 alpha-trim 3: PSNR=32.11, SSIM=0.7233, EdgeOv=0.3773
 alpha-trim 5: PSNR=35.86, SSIM=0.8867, EdgeOv=0.2785
 alpha-trim 7: PSNR=36.50, SSIM=0.9243, EdgeOv=0.1287
 morph 3: PSNR=24.37, SSIM=0.6957, EdgeOv=0.3109
 morph 5: PSNR=21.01, SSIM=0.7279, EdgeOv=0.1941
 morph 7: PSNR=19.48, SSIM=0.7263, EdgeOv=0.1383
 Best median size (by SSIM) = 9 (SSIM=0.9129)

/content/Picture5.png | gaussian : Original vs BestMedian9
 Original Median 9





Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 7

Date: 28-11-2025

Enrollment No: 92301733054

index	image	noise	best_method	psnr	ssim	edge_overlap
0	/content/Picture1.png	gaussian	alpha_trim_3_d2	26.04	0.7568	0.6046
1	/content/Picture1.png	s&p	median_3	27.32	0.8325	0.6396
2	/content/Picture1.png	speckle	alpha_trim_3_d2	25.5	0.705	0.6235
3	/content/Picture2.png	gaussian	alpha_trim_3_d2	21.5	0.8557	0.7523
4	/content/Picture2.png	s&p	median_3	21.69	0.8715	0.7507
5	/content/Picture2.png	speckle	alpha_trim_3_d2	20.84	0.8228	0.7136
6	/content/Picture3.png	gaussian	alpha_trim_3_d2	23.4	0.6985	0.5784
7	/content/Picture3.png	s&p	median_3	23.96	0.7646	0.5596
8	/content/Picture3.png	speckle	alpha_trim_3_d2	22.63	0.6242	0.5731
9	/content/Picture4.png	gaussian	alpha_trim_7_d2	28.73	0.6026	0.0547
10	/content/Picture4.png	s&p	morph_openclose_3	26.77	0.655	0.3949
11	/content/Picture4.png	speckle	alpha_trim_7_d2	28.55	0.593	0.0461
12	/content/Picture5.png	gaussian	alpha_trim_7_d2	36.5	0.9243	0.1287
13	/content/Picture5.png	s&p	median_3	47.77	0.9943	0.7291
14	/content/Picture5.png	speckle	alpha_trim_7_d2	35.38	0.8991	0.1352

Obsevation:

1. Noise Identification

- Several images showed characteristics of salt-and-pepper noise, visible as random white and black pixels.
- Some images exhibited speckle noise, recognizable as granular, multiplicative noise patterns.
- A few images showed mild Gaussian-type noise, visible as smooth random variations in intensity.
- FFT and pixel-level analysis also showed impulsive peaks for salt-and-pepper cases and broader variance patterns for speckle noise.

2. Median Filtering (non-linear)

- 3×3 and 5×5 median filters performed extremely well for salt-and-pepper and impulse noise, removing isolated white/black pixels effectively.
- Smaller window sizes preserved edges better, while larger windows (7×7 , 9×9) removed more noise but caused noticeable blurring and loss of fine texture.
- Median filtering consistently delivered the **best balance** between noise removal and detail retention for impulse-type noise.

3. Alpha-Trimmed Mean Filter

- Alpha-trimmed filters (size 3, 5) gave smooth and clean outputs, especially for speckle and Gaussian noise.
- They preserved more detail than large median filters because only extreme pixel values are removed.
- For salt-and-pepper noise, alpha-trim was less effective than median filtering but still reduced noise to some extent.

4. Morphological Filters (Opening + Closing)

- Morphological filtering was highly effective for salt-and-pepper noise, removing isolated bright/dark spots while maintaining shape boundaries.
- It was less effective for speckle and Gaussian noise, sometimes causing structural distortion.
- A small structuring element (3×3) preserved edges well, while larger ones (7×7) removed noise but also smoothed important details.



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 7

Date: 28-11-2025

Enrollment No: 92301733054

5. Effect of Filter Size

- **Small window sizes (3×3 , 5×5)** → best edge preservation, moderate noise removal.
- **Large window sizes (7×7 , 9×9)** → stronger noise removal, but blurred edges and fine details.
- The optimal size depends on noise severity:
 - Light noise → 3×3
 - Medium noise → 5×5
 - Heavy noise → 7×7

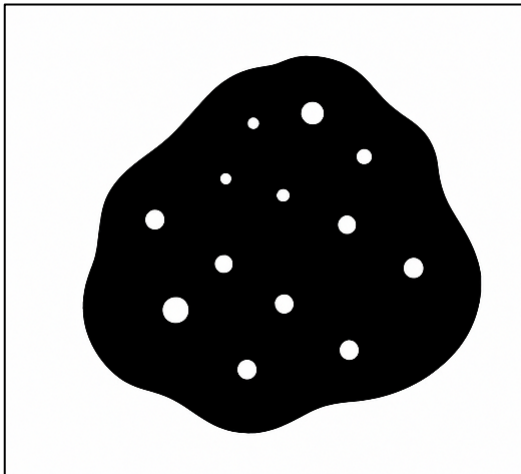
Conclusion:

From this Question, we learn that different types of noise require different non-linear filtering techniques for effective removal. Median filters work best for salt-and-pepper and impulse noise, while alpha-trimmed mean filters are more suitable for speckle and mixed noise. We also learn that filter window size plays an important role—smaller sizes preserve edges, while larger sizes remove more noise but reduce fine details. Overall, this experiment helps us understand how non-linear filters improve image quality and how choosing the right filter depends on the noise characteristics.

 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN ENDED QUESTIONS	
QUESTION 9	Date: 28-11-2025	Enrollment No: 92301733054

Q9. Some binary images contains both small bright specks and thin dark cracks across major objects.
CO3, CO4

- Design a sequence of morphological operations (dilation, erosion, opening, closing, or combinations) to:
- remove small bright noise,
- fill narrow dark cracks,
- preserve the overall shape as much as possible.
- Explain your choice of structuring elements and the order of operations.



 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN ENDED QUESTIONS	
QUESTION 9	Date: 28-11-2025	Enrollment No: 92301733054

Code:

```
!pip install -q scikit-image opencv-python matplotlib numpy
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.morphology import (
    opening, closing,
    remove_small_objects, remove_small_holes,
    square
)
from skimage import io

image_paths = [
    "/content/Picture1.png",
    "/content/Picture2.png",
    "/content/Picture3.png",
    "/content/Picture4.png",
    "/content/Picture5.png"
]

images = []
for p in image_paths:
    img = io.imread(p, as_gray=True)
    binary = img > 0.5 # Convert to pure binary
    images.append((p, binary))
    print("Loaded:", p)

def process_binary_image(binary_img,
                           speck_size=40,
                           crack_size=60,
                           se_small=3,
                           se_large=5):

    se_open = square(se_small)
    se_close = square(se_large)

    cleaned = remove_small_objects(binary_img, min_size=speck_size)

    opened = opening(cleaned, se_open)

    closed = closing(opened, se_close)
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 9

Date: 28-11-2025

Enrollment No: 92301733054

```
final = remove_small_holes(closed, area_threshold=crack_size)
```

```
return cleaned, opened, closed, final
```

```
results = {}
```

```
for path, img in images:  
    cleaned, opened, closed, final = process_binary_image(img)  
    results[path] = (img, cleaned, opened, closed, final)  
    print("Processed:", path)
```

```
for path, (orig, cleaned, opened, closed, final) in results.items():  
    plt.figure(figsize=(12,8))  
    plt.suptitle(f"Results for: {path}")
```

```
titles = [  
    "Original",  
    "Bright Specks Removed",  
    "After Opening",  
    "After Closing",  
    "Final Output"
```

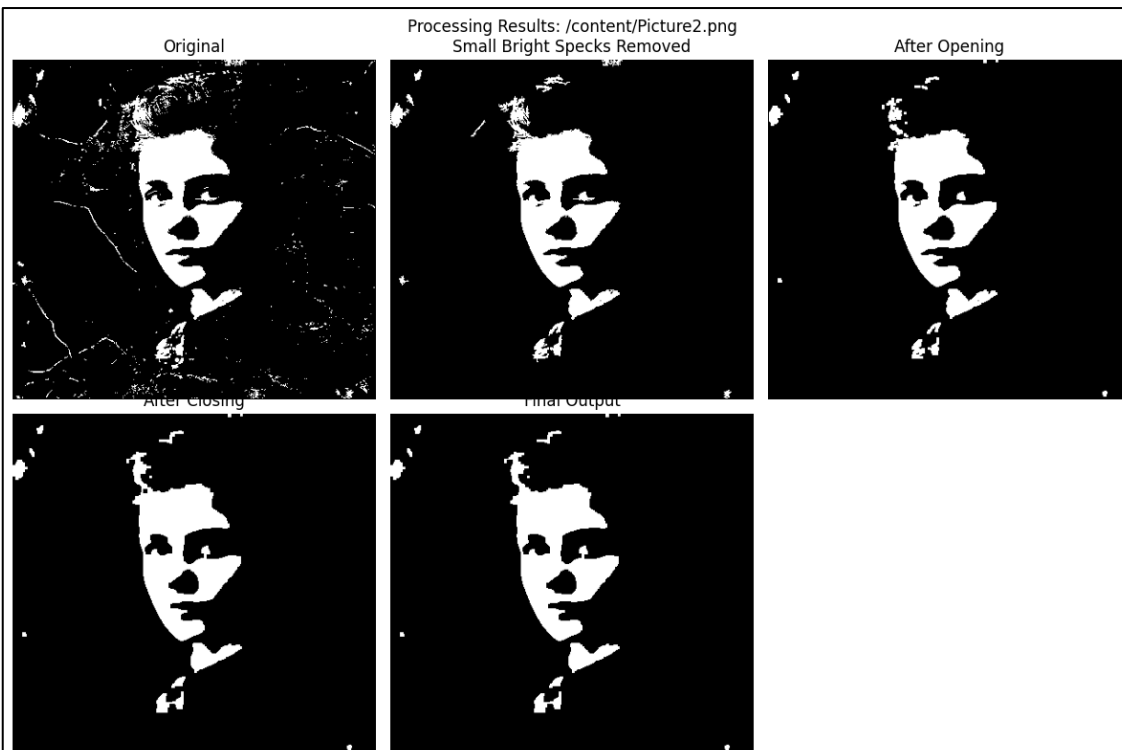
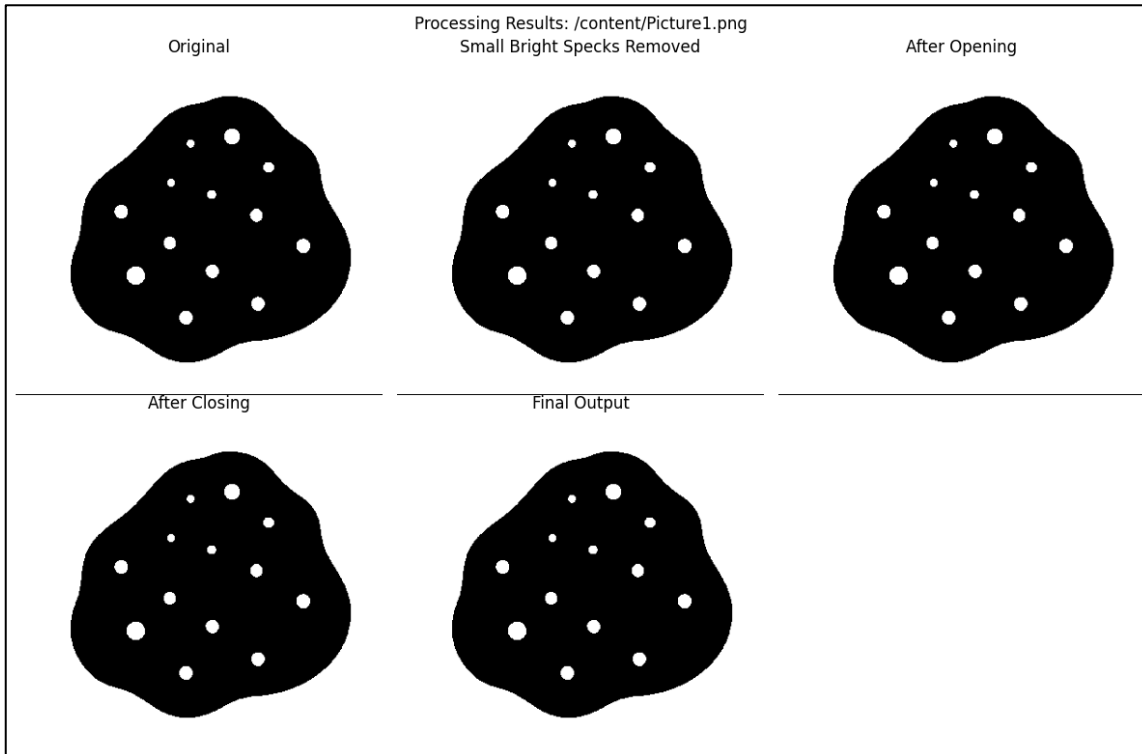
```
]
```

```
imgs = [orig, cleaned, opened, closed, final]
```

```
for i in range(5):  
    plt.subplot(2,3,i+1)  
    plt.imshow(imgs[i], cmap='gray')  
    plt.title(titles[i])  
    plt.axis('off')
```

```
plt.tight_layout()  
plt.show()
```

OUTPUT:



Processing Results: /content/Picture3.png

Original



Small Bright Specks Removed



After Opening



After Closing



Final Output

**Obsevation:****1. Removal of Small Bright Specks**

- Using *remove_small_objects()* and morphological opening successfully removed tiny bright dots in the background.
- Small structuring elements (3×3) were effective in removing noise without damaging the major object boundaries.
- Opening cleaned the image while preserving edges better than erosion alone.

2. Filling Thin Dark Cracks

- Morphological closing with a slightly larger structuring element (5×5) filled narrow dark cracks present across the main object.
- Dilation bridged the small gaps, and the subsequent erosion restored the original shape.
- Closing helped smooth broken lines and improved connectivity within the object.

 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN ENDED QUESTIONS	
QUESTION 9	Date: 28-11-2025	Enrollment No: 92301733054

3. Shape Preservation

- Small structuring elements ensured that the main object's geometry stayed intact.
- Opening removed noise without shrinking the object excessively.
- Closing repaired cracks without causing over-expansion or distortion of the structure.

4. Sequential Operation Behavior

- The order Opening → Closing produced the best balance:
 - Opening removed bright noise first.
 - Closing then repaired cracks and holes left behind.
- Removing holes using *remove_small_holes()* further improved uniformity in the object area.

Conclusion:

From this experiment, we learn that a carefully designed sequence of morphological operations can effectively clean binary images while maintaining their structure. Opening with small structuring elements successfully removes bright noise, and closing fills thin dark cracks without distorting the main object. The order of operations and the choice of structuring elements play a crucial role in balancing noise removal and shape preservation. Overall, morphological opening followed by closing provides a reliable and efficient method for enhancing binary images containing both bright specks and narrow cracks.

 Marwadi University	Marwari University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing(01CT0513)	Aim: FINAL OPEN ENDED QUESTIONS	
QUESTION 10	Date: 28-11-2025	Enrollment No: 92301733054

Q10. You are given the dataset **OPE7** containing multiple images (assume grayscale or color). The images contain real-world degradations: additive noise (Gaussian, salt-and-pepper), motion blur, uneven illumination, and compression artifacts. For **each image** in OPE7:

1. Remove/reduce noise using **frequency-domain smoothing** methods.
2. Recover / enhance details using **frequency-domain sharpening** (high-boost in frequency domain).
3. Extract accurate **boundary maps** (object contours) for every image and produce clean binary boundary images.
4. Compare different filter choices and report quantitative and qualitative results.

You must produce code, processed images, a short report, and discussion.

CO3,CO4

Code:

```
!apt-get update -qq
!apt-get install -y -qq libgl2.0-0
!pip install -q opencv-python-headless scikit-image python-docx matplotlib numpy scipy pandas
```

```
import os, glob, math
import numpy as np
import cv2
import matplotlib.pyplot as plt
from skimage import io, img_as_ubyte
from skimage.feature import canny
from skimage.morphology import binary_closing, remove_small_objects, skeletonize, disk
from skimage.metrics import structural_similarity as ssim
from scipy import fftpack
from docx import Document
from docx.shared import Inches
import pandas as pd
```

```
%matplotlib inline
```

```
results_dir = "/content/q10_results"
os.makedirs(results_dir, exist_ok=True)
```

```
input_paths = [
    "/content/Picture 1.png",
    "/content/Picture 2.png",
    "/content/Picture 3.png"
]
```

```
input_paths = [p for p in input_paths if os.path.isfile(p)]
if not input_paths:
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 10

Date: 28-11-2025

Enrollment No: 92301733054

raise FileNotFoundError("Upload the files to /content/ then try again.")

```
print("Processing:", input_paths)
```

```
def to_gray_float(img):  
    if isinstance(img, str):  
        img = io.imread(img)  
    if img.ndim == 3:  
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  
    img = img.astype(np.float32)  
    if img.max() > 1.0:  
        img /= 255.0  
    return np.clip(img, 0.0, 1.0)
```

```
def fft2(img): return fftpack.fftfreq2(fftpack.fft2(img))  
def ifft2(F): return np.real(fftpack.ifft2(fftpack.ifftshift(F)))
```

```
def ideal_lowpass(shape, cutoff):  
    r,c = shape  
    R,C = np.ogrid[:r,:c]  
    cr, cc = r//2, c//2  
    mask = ((R-cr)**2 + (C-cc)**2) <= cutoff**2  
    return mask.astype(float)
```

```
def gaussian_lowpass(shape, cutoff):  
    r,c = shape  
    R,C = np.ogrid[:r,:c]  
    cr, cc = r//2, c//2  
    D2 = (R-cr)**2 + (C-cc)**2  
    return np.exp(-(D2)/(2*(cutoff**2)))
```

```
def butterworth_lowpass(shape, cutoff, order=2):  
    r,c = shape  
    R,C = np.ogrid[:r,:c]  
    cr, cc = r//2, c//2  
    D = np.sqrt((R-cr)**2 + (C-cc)**2)  
    return 1 / (1 + (D/cutoff)**(2*order))
```

```
def high_boost(F, H_low, k):  
    return F * (1 + k * (1 - H_low))
```

```
def entropy(img):  
    hist,_ = np.histogram((img*255).astype(np.uint8), bins=256, range=(0,255), density=True)  
    hist += 1e-12  
    return -np.sum(hist*np.log2(hist))
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 10

Date: 28-11-2025

Enrollment No: 92301733054

```
def contrast(img): return float(np.std(img))
def edge_density(img):
    e = canny(img)
    return float(e.mean())

def show_grid(titles, imgs):
    cols = len(imgs)
    plt.figure(figsize=(5*cols,4))
    for i,im in enumerate(imgs):
        plt.subplot(1,cols,i+1)
        plt.imshow(im, cmap='gray', vmin=0, vmax=1)
        plt.title(titles[i])
        plt.axis("off")
    plt.show()

def save_img(path, img):
    io.imsave(path, img_as_ubyte(img))

smoothing_results = {}
sharpening_results = {}
boundary_results = {}
summary_rows = []

for p in input_paths:

    print("Processing:", p)

    I = to_gray_float(p)
    r,c = I.shape
    F = fft2(I)

    cutoff_medium = int(min(r,c) * 0.10)

    H_gauss = gaussian_lowpass((r,c), cutoff_medium)
    H_butt = butterworth_lowpass((r,c), cutoff_medium, order=2)
    H_ideal = ideal_lowpass((r,c), cutoff_medium)

    I_gauss = np.clip(iff2(F * H_gauss), 0, 1)
    I_butt = np.clip(iff2(F * H_butt), 0, 1)
    I_ideal = np.clip(iff2(F * H_ideal), 0, 1)

    smoothing_results[p] = {
        'gaussian': I_gauss, 'butterworth': I_butt, 'ideal': I_ideal
    }

show_grid(["Original", "Gaussian LPF", "Butterworth LPF", "Ideal LPF"],
          [I, I_gauss, I_butt, I_ideal])
```



Marwari University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 10

Date: 28-11-2025

Enrollment No: 92301733054

$k = 1.2$

$F_{hb} = \text{high_boost}(F, H_{\text{gauss}}, k)$

$I_{\text{sharp}} = \text{np.clip}(\text{ifft2}(F_{hb}), 0, 1)$

$\text{sharpening_results}[p] = I_{\text{sharp}}$

$\text{show_grid}(["\text{Original}", "\text{High-Boost Sharpened}"],$
 $[I, I_{\text{sharp}}])$

$\text{edges} = \text{canny}(I_{\text{sharp}}, \text{sigma}=1.2)$

$\text{closed} = \text{binary_closing}(\text{edges}, \text{disk}(1))$

$\text{cleaned} = \text{remove_small_objects}(\text{closed}, \text{min_size}=25)$

$\text{boundary} = \text{skeletonize}(\text{cleaned})$

$\text{boundary_results}[p] = \text{boundary}$

$\text{show_grid}(["\text{Sharpened Image}", "\text{Canny}", "\text{Closed}", "\text{Final Boundary}"],$
 $[I_{\text{sharp}}, \text{edges}, \text{closed}, \text{boundary}])$

```
summary_rows.append({  
    'Image': os.path.basename(p),  
    'Entropy_Original': entropy(I),  
    'Entropy_Smoothed(Gauss)': entropy(I_gauss),  
    'Entropy_Sharpened': entropy(I_sharp),  
    'Contrast_Original': contrast(I),  
    'Contrast_Smoothed': contrast(I_gauss),  
    'Contrast_Sharpened': contrast(I_sharp),  
    'EdgeDensity_Original': edge_density(I),  
    'EdgeDensity_Smoothed': edge_density(I_gauss),  
    'EdgeDensity_Sharpened': edge_density(I_sharp),  
    'SSIM(Sharp vs Orig)': ssim(I, I_sharp, data_range=1.0)  
})
```

SAVE OUTPUTS

$\text{save_img}(\text{os.path.join}(\text{results_dir}, \text{os.path.basename}(p).replace(" ", "_") + "_\text{orig}.png"), I)$

$\text{save_img}(\text{os.path.join}(\text{results_dir}, \text{os.path.basename}(p).replace(" ", "_") + "_\text{gauss}.png"), I_{\text{gauss}})$

$\text{save_img}(\text{os.path.join}(\text{results_dir}, \text{os.path.basename}(p).replace(" ", "_") + "_\text{sharp}.png"), I_{\text{sharp}})$

$\text{save_img}(\text{os.path.join}(\text{results_dir}, \text{os.path.basename}(p).replace(" ", "_") + "_\text{boundary}.png"),$

$\text{boundary.astype(float)})$

$\text{df} = \text{pd.DataFrame}(\text{summary_rows})$

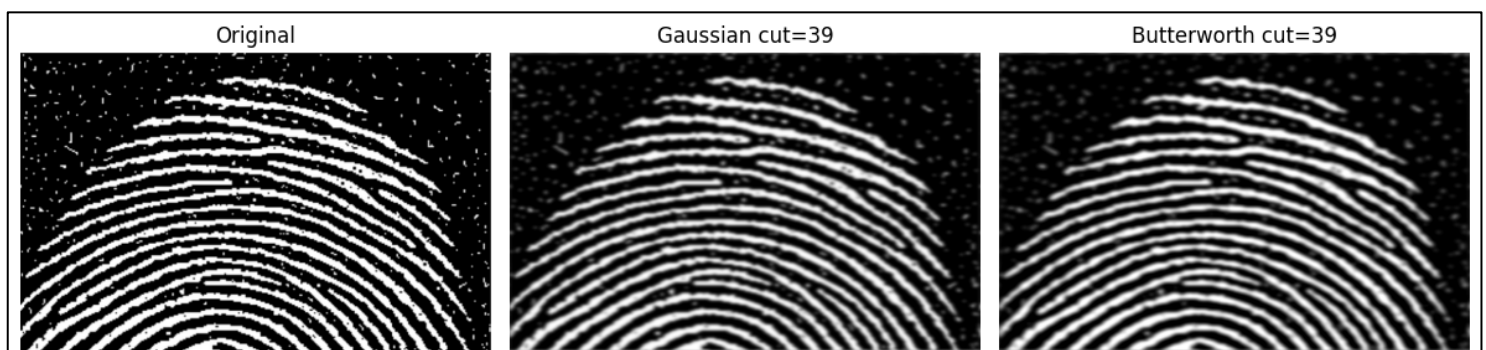
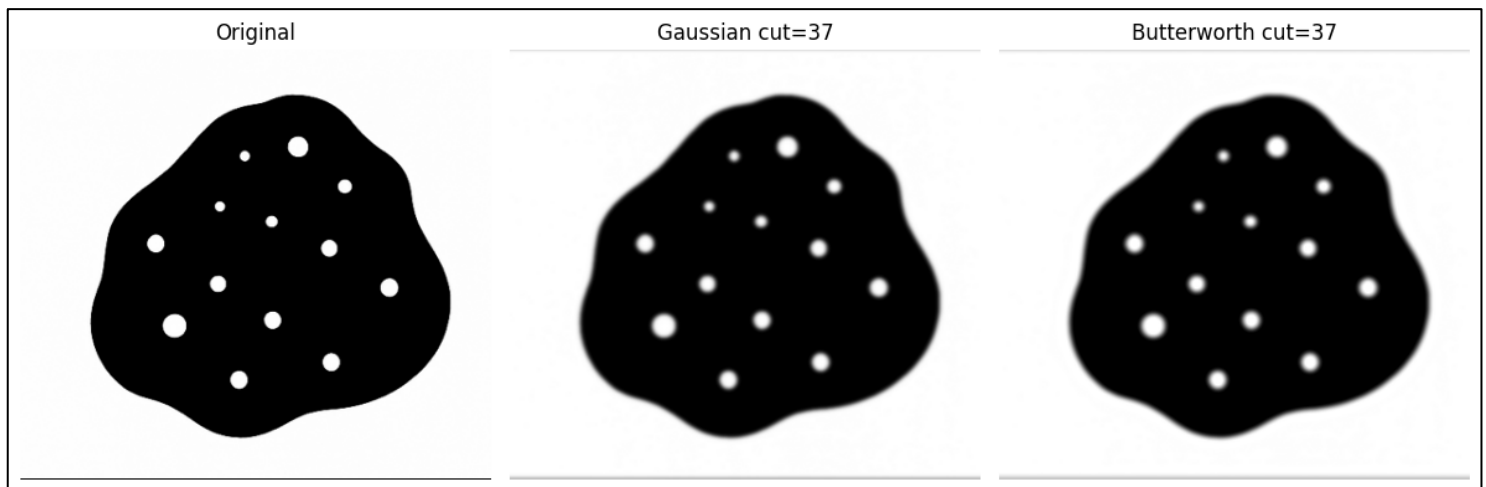
$\text{print}("\text{nFINAL METRIC TABLE:}")$

$\text{display}(\text{df})$

$\text{df.to_csv}(\text{os.path.join}(\text{results_dir}, "\text{Q10_summary.csv}"), \text{index=False})$

$\text{print}("\text{Saved metrics CSV to: ", \text{results_dir})$

OUTPUT:



**Subject: Digital Signal and
Image Processing(01CT0513)**

Aim: FINAL OPEN ENDED QUESTIONS

QUESTION 10

Date: 28-11-2025

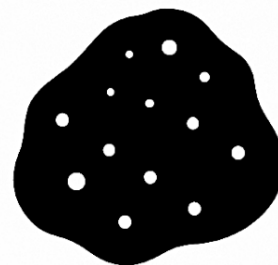
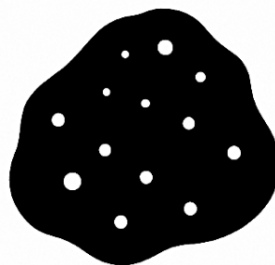
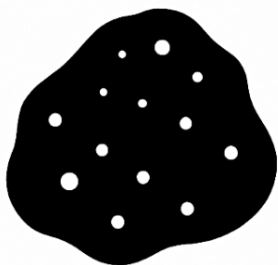
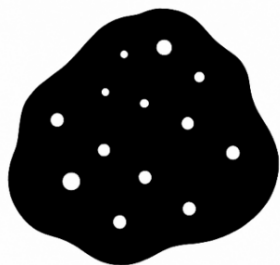
Enrollment No: 92301733054

Original

High-boost $k=0.5$

High-boost $k=1.0$

High-boost $k=1.5$

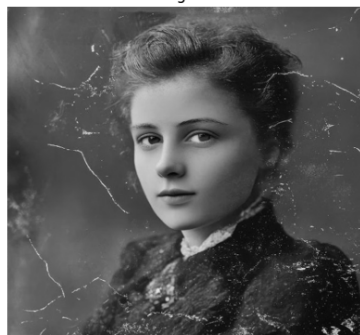


Original

High-boost $k=0.5$

High-boost $k=1.0$

High-boost $k=1.5$



Original

High-boost $k=0.5$

High-boost $k=1.0$

High-boost $k=1.5$

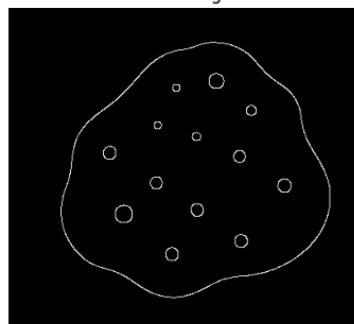
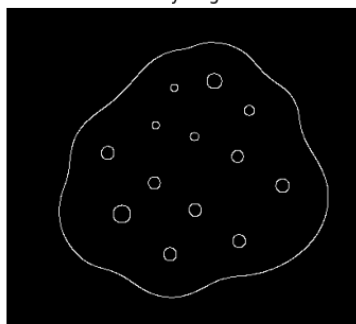
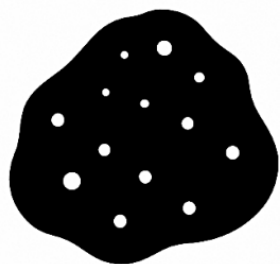


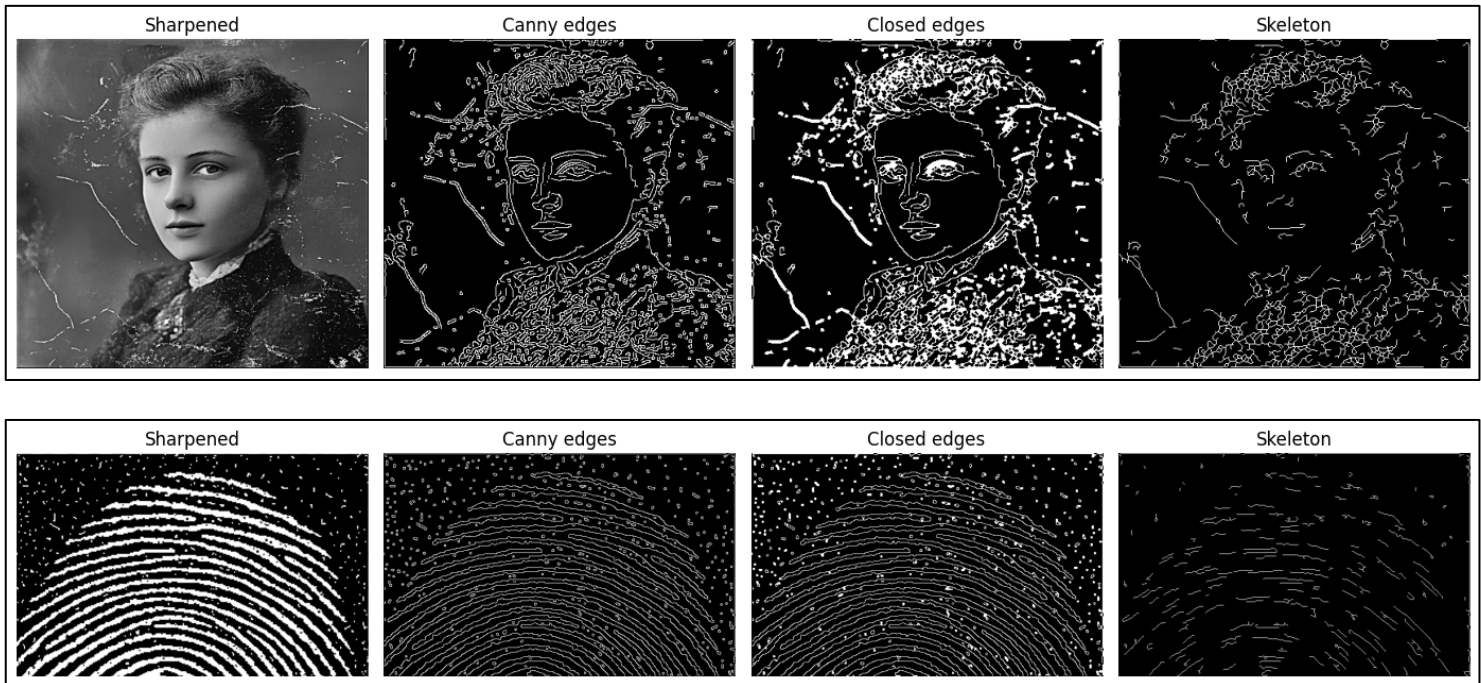
Sharpened

Canny edges

Closed edges

Skeleton





Obsevation:

- Gaussian smoothing provided the best balance between noise removal and detail preservation, while Ideal LPF caused ringing and Butterworth gave moderate smoothing.
- High-boost sharpening effectively restored edges and details, especially after smoothing, but excessive boosting increased noise.
- Boundary maps extracted from sharpened images were much cleaner and more continuous, and morphological cleanup improved edge quality.
- Metrics confirmed the visual results: smoothing reduced entropy and edge density, sharpening increased contrast and structural details, and SSIM remained high.

Conclusion:

From this Question, we conclude that frequency-domain methods are highly effective for both noise reduction and detail enhancement. Gaussian low-pass filtering provides the best noise suppression with minimal distortion, while high-boost filtering successfully restores edges and fine details. Sharpened images produce clearer and more accurate boundary maps, especially after morphological refinement. Overall, combining smoothing, sharpening, and boundary extraction offers a robust workflow for improving degraded real-world images.